

Master's thesis
Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 7, 2026

Supervisor(s): Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Examiner(s):

University of Helsinki
Faculty of Science

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki

HELSINGIN YLIOPISTO — HELSINGFORS UNIVERSITET — UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Degree programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidantal — Number of pages
Master's thesis	August 7, 2026	105

Tiivistelmä — Referat — Abstract

Context. Radio Access Network (RAN) control-plane software is written mostly in C++ for performance, but C++ carries well-known memory-safety and concurrency risks. Rust promises comparable performance with safety guaranteed at compile time, yet its fit for latency-critical telecom control planes is unproven. The work is done at Nokia on a subsystem modelled on a real gNB-CU-CP component.

Objective. The thesis compares modern C++ and Rust on a UE Context Management service. As similar average throughput is already known, it targets four less-explored dimensions: tail latency and jitter under overload; which C++ defect classes Rust prevents at compile time; the overhead of Rust's safety model; and the cost of incremental adoption in a C++ codebase.

Methods. A language-agnostic specification is frozen first; two functionally equivalent implementations are then built with identical algorithms, concurrency model, allocator, and build settings. A synthetic workload is swept to overload, both versions pass the same parity tests, and evaluation adds a fault-injection study and an FFI prototype.

Results. Safety and migration results are final. All five injected memory-safety and data-race defect classes are prevented at compile time in safe Rust; in C++ each compiles and is caught, if at all, only at runtime and only under the right sanitiser. Two boundary classes, a lock-order deadlock and a lost update, compile and misbehave in both. The FFI seam keeps the core `unsafe`-free at a cost under 1.37% of one operation. On performance, an end-to-end operation proves to be 0.040% work and the rest cross-thread handoff, so the sweep bounds the design, not the language: measured apart, Rust's operation is twice as fast, its process a few MiB larger. The tail awaits the specified testbed.

Conclusions. The thesis delivers a reproducible benchmarking framework, a parity-tested pair of implementations, a defect-class taxonomy with its boundary, and a quantified migration cost. The barrier to adopting Rust here is engineering effort and the learning curve rather than run-time performance, and what is gained is freedom from data races rather than from concurrency defects generally.

ACM Computing Classification System (CCS):
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, FFI

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — Övriga uppgifter — Additional information

Software study track

Contents

1 Introduction **1**

1.1 Key terms and abbreviations 2
1.2 Motivation and industrial context 2
1.3 Problem statement 5
1.4 Research questions 5
1.5 Research approach at a glance 6
1.6 Contributions, results, and limitations 6
1.7 Significance and beneficiaries 7
1.8 Thesis outline 8

2 Background: Systems Languages for Latency-Critical Software **9**

2.1 RAN control-plane systems and the gNB-CU-CP 9
2.2 Latency, throughput, and predictability in real-time software 10
2.3 C++ for latency-critical systems 11
2.4 The Rust ownership and safety model 13
2.5 The systems-language landscape 14
2.6 Concurrency models for control-plane software 15
2.7 Interoperability between C++ and Rust 17
2.8 Research gap 18

3 Problem Context: UE Context Management and Predictability **21**

3.1 The UE Context Management subsystem 21
3.2 Memory-safety and concurrency defect classes 22
3.3 Tail latency and predictability in RAN 24

4 Objectives **27**

4.1 Deficiencies in existing comparisons 27
4.2 Solution objectives 28
4.3 Hypotheses 28
4.4 Success criteria 29

4.5 Scope and boundaries . . . 29

5 Research Methodology 31
5.1 Research approach . . . 31
5.2 A constructive, comparative experiment . . . 31
5.3 Methods for the four research questions . . . 33
5.3.1 Performance measurement under a load sweep (RQ1, RQ3) . . . 33
5.3.2 Controlled fault injection (RQ2) . . . 34
5.3.3 Foreign-function-interface migration case study (RQ4) . . . 36
5.4 Fairness as a controlled-variable strategy . . . 36
5.5 Statistical treatment of measurements . . . 37
5.6 Reproducibility and research artifacts . . . 39
5.7 Ethical and data-protection considerations . . . 39

6 Design and Implementation 41
6.1 Shared specification: the fairness baseline . . . 41
6.2 Conceptual model and concurrency design . . . 41
6.3 Architectural choices and technology stack . . . 42
6.4 Implementation details . . . 43
6.4.1 The UE context and its memory layout . . . 43
6.4.2 The state machine as implemented . . . 44
6.4.3 Shard selection and the intra-shard store . . . 45
6.4.4 The bounded queue . . . 46
6.4.5 Requests, responses, and the cost of a tagged struct . . . 46
6.4.6 Service lifecycle, shutdown, and ownership . . . 48
6.4.7 Forced idioms and what they cost . . . 49
6.4.8 The safety argument for the two cores . . . 52
6.5 Public API and operation semantics . . . 53
6.6 Request lifecycle and the completion handoff . . . 53
6.7 Parity verification . . . 55

7 Experimental Setup and Evaluation 57
7.1 Testbed configuration . . . 57
7.2 Workload and evaluation measurements . . . 58
7.3 Data collection and metrics . . . 59
7.4 Evaluation data . . . 59
7.5 Evaluation results . . . 59
7.5.1 Performance (RQ1, RQ3) . . . 60
7.5.2 What the end-to-end number is made of . . . 62

7.5.3 Latency, throughput, and saturation across the sweep 66
7.5.4 Resource use (RQ3) 67
7.5.5 Safety via fault injection (RQ2) 69
7.5.6 Migration (RQ4) 73

8 Discussion **77**
8.1 Summary of findings 77
8.2 Discussion on the research questions 77
8.3 Revisiting the hypotheses 82
8.4 Engineering guidelines for incremental adoption 83
8.5 Related work 85
8.6 Threats to validity and mitigations 88
8.7 Ethical and professional reflection 92

9 Conclusion and Future Work **93**

Bibliography **97**

Appendix A Fault-injection example **105**

1. Introduction

The mobile network is undergoing a profound softwarization. Functions that were once delivered by fixed-function, vertically integrated hardware are being re-implemented as software running on commodity servers, and the Radio Access Network (RAN), long the most specialised part of the system, is no longer an exception. In the 5G NG-RAN architecture the base station (gNB) is disaggregated into a central unit and one or more distributed units, and the central unit is split again into control- and user-plane functions [1, 2]. Open-RAN initiatives push this disaggregation further, turning the RAN into virtualised, multi-vendor software connected by open interfaces [48]. The resulting control plane, the gNB-CU-CP, is a large, stateful, highly concurrent software system. It manages the lifecycle of every connected device, drives the RRC signaling procedures [4], and coordinates mobility and handovers, all under strict latency budgets. The rates involved are high: a single busy site drives on the order of thousands of connection-management and mobility procedures per second, each bounded by a budget measured in milliseconds [5], while a deployed control plane coordinates millions of simultaneously connected devices. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing behaviour of this software increasingly determine the quality of the network as a whole. An outage or a tail-latency regression here is not a localised defect; it is a degradation felt across every device the site serves [18, 12].

This control-plane software is, today, written predominantly in C++. The choice is a rational one: C++ offers the low-level control, mature tooling, and performance that latency-critical systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [41, 59]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time, with performance intended to be competitive with C++ [33, 29]. Whether that promise holds for a telecom-grade, latency-critical

control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, addresses the question directly: it develops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service modelled on a real Nokia gNB-CU-CP component, in modern C++ and in Rust, and compares them under a realistic synthetic signaling workload.

1.1 Key terms and abbreviations

For readers outside the telecom domain, Table 1.1 fixes the abbreviations and terms used throughout the thesis. Three of them recur constantly: the *gNB-CU-CP* is the control-plane software this work is about; *UE Context Management (UECM)* is the specific subsystem reimplemented and measured; and *tail latency* (the p99/p99.9 percentiles rather than the mean) is the property that, more than average speed, decides whether such a system meets its deadlines [18, 12].

1.2 Motivation and industrial context

Two trends make this comparison timely. The first is the cloud-native evolution of the RAN itself. Moving control-plane functions onto general-purpose servers raises, rather than lowers, the premium on software quality: the software is longer-lived, evolves continuously, runs at higher concurrency, and must meet its deadlines without the determinism that purpose-built hardware once provided. This shift is accelerated by Open-RAN disaggregation, which turns the base station into virtualised, multi-vendor software connected by open interfaces [48], so that the control plane a vendor ships increasingly competes on software-engineering quality (reliability, predictable timing, security, and the pace at which a large codebase can be evolved safely) rather than on bespoke hardware. In this setting a latency regression or a memory-corruption defect is not a localised bug but a network-wide reliability and security risk, and a language-selection question stops being academic and becomes a concrete engineering decision. The second trend is an industry-wide shift toward memory-safe languages. Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them [41, 59, 52]. This concern has risen to the level of national policy: cybersecurity agencies now urge software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ [17, 45]. Rust in particular has moved from a niche language into operating-system kernels, cloud infrastructure, and increasingly into telecommunications; production hypervisors such

Table 1.1: Abbreviations and key terms used throughout the thesis.

Term	Meaning
RAN	Radio Access Network; the part of the mobile network connecting devices to the core
NG-RAN	Next-Generation RAN, the 5G radio-access-network architecture
gNB	5G base station (“next-generation NodeB”)
gNB-CU / DU	Central Unit / Distributed Unit; the disaggregated halves of a gNB
gNB-CU-CP	Control-plane part of the central unit; the system class studied here
UE	User Equipment; an end device (phone, modem) attached to the network
UECM	UE Context Management; the per-UE state-management subsystem reimplemented here
RRC	Radio Resource Control; the control-plane signaling protocol between UE and gNB
DRB	Data Radio Bearer; a per-UE user-plane data channel set up during a session
Control plane	The signaling and management path, as opposed to the user-plane data path
Tail latency	High-percentile latency (p99, p99.9), the worst-case-oriented timing metric
FFI	Foreign Function Interface; calling Rust from C++ across a C ABI
HDR histogram	High-Dynamic-Range histogram, used for accurate latency capture

as AWS Firecracker, which underpins its serverless platform, are already written in Rust [6]. Yet the evidence base for Rust's suitability rests largely on general-purpose software and on average-case performance benchmarks. The specific question a telecom vendor must answer, whether Rust can meet the tail-latency and predictability requirements of a RAN control plane, whether it genuinely prevents the defect classes that plague the existing C++ code, and what it costs to adopt it incrementally inside a large C++ codebase, has not been answered with direct, controlled evidence. Providing such evidence for a representative subsystem is the motivation for this work.

Industrial setting. This thesis is carried out at Nokia, one of the small number of vendors that build, maintain, and evolve production RAN software end to end, and the subsystem it studies, UE Context Management (UECM), is modelled on a real gNB-CU-CP control-plane component rather than a textbook example. The gNB-CU-CP is not a peripheral module: in the 3GPP disaggregated architecture it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle while delegating user-plane bearer handling to the CU-UP over the E1 interface [1, 2, 3, 4], all under the millisecond-scale procedure budgets the 5G system places on control signaling [5] and at the very high event rates and connected-device counts a deployed network sustains. Because it is written in C++, it inherits both that language's performance and its memory-safety and concurrency hazards, which is precisely where the two trends above converge. Software of this class is shipped, evolved, and operated across live operator networks, so a language-selection decision here is a concrete production concern rather than a hypothetical one, which is what makes a controlled comparison on a representative subsystem valuable to a vendor. UECM is a deliberate choice of study object within that setting: it is small enough to reimplement faithfully and to reason about in full, yet it exercises exactly the properties that make RAN control-plane code hard: per-UE mutable state, a strict operation lifecycle, high concurrency across many UEs, and hard tail-latency expectations. Conclusions drawn from it therefore transfer to the broader class of stateful, concurrent control-plane services, while remaining reproducible by readers outside Nokia because the implementation is driven entirely by a public, frozen specification rather than by proprietary code.

A concrete failure mode. To make the stakes concrete, consider a mobility storm in which many UEs hand over at once. A single mishandled pointer, a UE context freed on release while an in-flight signaling message still references it, produces a use-after-free. In C++ this compiles cleanly and may pass ordinary testing, only to corrupt an unrelated UE's state or crash the worker thread under precisely the high-concurrency conditions the system exists to handle; the same latent race can equally manifest as

an unpredictable tail-latency spike rather than an outright crash. In safe Rust the same design does not compile: ownership and borrowing reject the dangling reference at build time. This one scenario captures why the thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload) rather than average throughput alone.

1.3 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation, which are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [18], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages, yet for the RAN control plane they lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput), whether Rust's compile-time guarantees actually eliminate the specific defect classes that afflict the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [11, 51], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

1.4 Research questions

The thesis is organised around four research questions, ordered to put predictability and migration ahead of raw average performance:

RQ1 How do Rust and modern C++ compare in latency predictability (mean, p99, p99.9 tail latency, and jitter) for a RAN control-plane workload under increasing

load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, FFI interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

1.5 Research approach at a glance

The four questions are answered constructively rather than by literature synthesis. The core artifact is a pair of functionally equivalent UECM implementations, one in modern C++ and one in Rust, built from a single frozen specification, so that any measured difference is attributable to the language and its idioms rather than to divergent designs. RQ1 and RQ3 are answered by driving both implementations with an identical, deterministic synthetic signaling workload in a load sweep from light load through saturation, capturing latency with matched high-dynamic-range histograms and resource use through the operating system, and comparing them with nonparametric confidence intervals [25]. RQ2 is answered by controlled fault injection: a catalogue of representative C++ defects is written deliberately and each is confronted with its Rust counterpart to record whether it is rejected at compile time, caught only at runtime, or missed entirely. The catalogue deliberately spans both defect classes Rust is expected to prevent and classes it is expected not to, so that the study can locate the boundary of the guarantee rather than only confirm it. RQ4 is answered by building a working C++-to-Rust FFI prototype and measuring the size of the interoperability seam, the scope of `unsafe`, and the per-call crossing cost. The full protocol (testbed, workloads, and statistical treatment) is given in Chapter 5.

1.6 Contributions, results, and limitations

The thesis contributes (i) a reproducible, telecom-relevant benchmarking framework for control-plane workloads; (ii) a frozen shared specification and two parity-tested implementations of a RAN control-plane subsystem in C++ and Rust; (iii) a quantitative comparison emphasising tail latency and predictability alongside average throughput;

(iv) a defect-class taxonomy obtained through controlled fault injection, including the boundary at which Rust's compile-time guarantee stops; and (v) evidence-based engineering and migration guidelines for language selection in cloud-native RAN evolution (Section 8.4). The headline findings are as follows. On *safety* (RQ2), every injected memory-safety and data-race defect class is prevented at compile time in safe Rust, whereas in C++ each compiles and is caught, if at all, only at runtime and only with the right sanitiser, while two deliberately chosen boundary classes are prevented by neither language; this result is final. On *migration* (RQ4), the FFI seam is small, keeps the core free of `unsafe`, and costs at most 1.37% of one operation when measured against that same operation called natively, so the real adoption cost is API design, not run-time overhead; this result is also final. On *performance* (RQ1, RQ3), measurements collected on a shared development environment show the two implementations indistinguishable end to end at the median and unresolvable in the tail. That null is less informative than it looks, and saying why is part of the contribution: an end-to-end operation here is 0.040% the work the language performs and the rest a cross-thread handoff the operating system performs, so the sweep bounds the design rather than the language. Measured separately, the operation itself is about twice as fast in Rust, for reasons traceable to the standard library's hash-map layout rather than to the safety model, while the Rust process carries a few MiB more resident memory under both allocators. The main limitations are the use of a synthetic workload, a single simplified subsystem, and single-author implementation bias; these are treated in detail in Chapter 8.

1.7 Significance and beneficiaries

The results are intended to be directly actionable. For RAN vendors and cloud-native network operators, the thesis supplies controlled evidence on the question that gates any language decision. That question is not "is Rust fast on average?" but "does Rust hold its tail latency and predictability under overload, and what does incremental adoption cost?", answered for a subsystem representative of production control-plane code. For the standards and policy discussion around memory-safe roadmaps [17, 45], it grounds a general mandate in a concrete telecom data point. For the research community, the frozen specification, the parity-tested implementations, the reproducible benchmarking framework, and the defect-class taxonomy form a reusable baseline for further language and systems studies. And for practising engineers, the fault catalogue and the migration prototype double as worked examples of what Rust's guarantees do and do not buy in this domain.

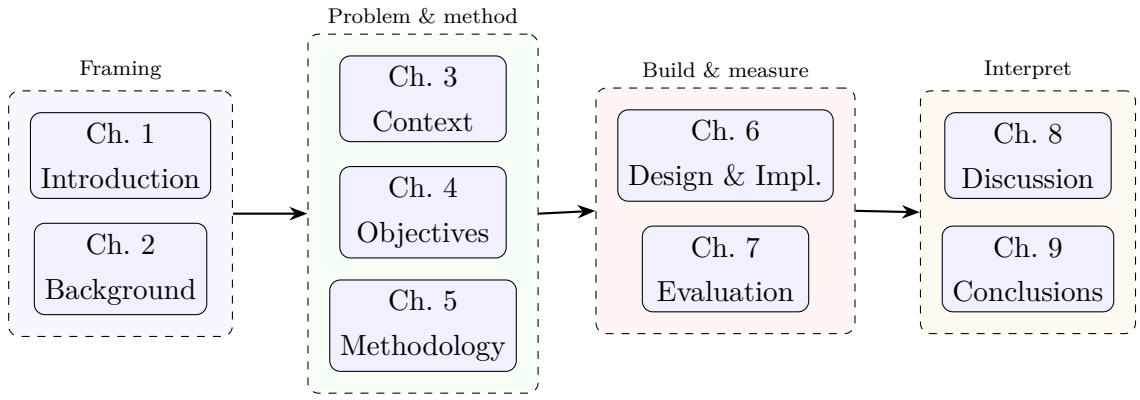


Figure 1.1: Roadmap of the thesis. The chapters progress from framing the problem, through the problem context, the objectives it sets, and the method by which they are pursued, to the constructed artifacts and their evaluation, and finally to interpretation and conclusions.

1.8 Thesis outline

Chapter 2 gives the background on RAN control-plane software, C++, and the Rust safety model. Chapter 3 describes the problem context: the UE Context Management subsystem, the relevant defect classes, and tail-latency predictability. Chapter 4 sets the objectives, hypotheses, scope, and boundaries, and Chapter 5 states the research approach by which they are pursued. Chapter 6 presents the design and implementation of both versions. Chapter 7 describes the experimental setup and reports the evaluation. Chapter 8 discusses the findings, related work, and threats to validity. Chapter 9 concludes and outlines future work. Figure 1.1 summarises this structure at a glance.

2. Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

The NG-RAN architecture splits the gNB into a central unit (gNB-CU) and distributed units, with the control plane (gNB-CU-CP) handling RRC and connection management for each UE [1, 2, 4]. Such control-plane systems must satisfy demanding non-functional requirements: very high event rates (UE mobility, signaling storms), strict and predictable latency bounds that the 5G service specifications quantify as KPIs [5], long-lived software with continuously evolving complexity, and heavily concurrent workloads. In these systems the worst-case and tail behaviour, rather than the average, determines whether deadlines are met, which motivates the predictability focus of this thesis.

Functionally, the gNB is disaggregated along two orthogonal axes. The first is a *central/distributed* split: a central unit (gNB-CU) hosts the higher layers of the protocol stack, while one or more distributed units (gNB-DU) host the lower, more time-critical layers close to the radio. The second is a *control/user-plane* split inside the central unit, separating the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts communicate over standardised interfaces: NG towards the 5G core, F1 between the CU and the DU, and E1 between the CU-CP and the CU-UP [1, 2, 3]. Open-RAN initiatives standardise these interfaces further so that each part can be supplied by a different vendor and run as software on commodity hardware [48]. Figure 2.1 places the subsystem studied in this thesis, UE Context Management (UECM), inside the gNB-CU-CP: it owns the per-UE control-plane state and mediates the RRC signaling and mobility procedures, which is why its correctness and timing behaviour are representative of the control plane as a whole.

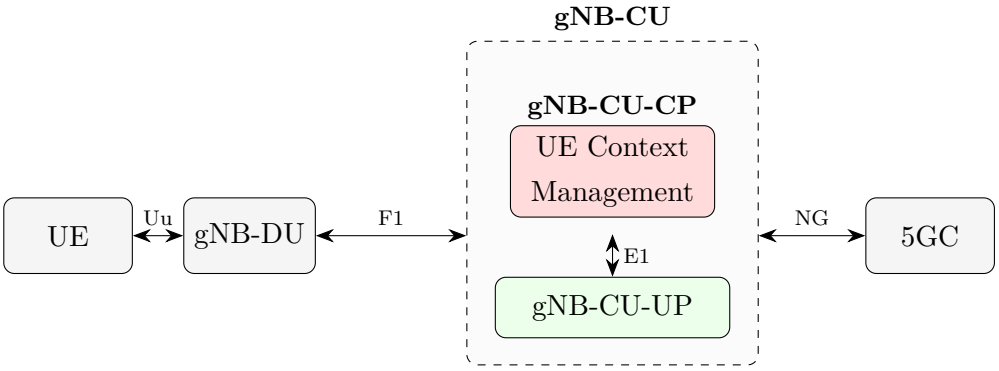


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. The gNB is split into a distributed unit (gNB-DU) and a central unit (gNB-CU), and the central unit is split again into control-plane (gNB-CU-CP) and user-plane (gNB-CU-UP) functions, connected by the standardised F1, E1, and NG interfaces. UE Context Management (highlighted) is the per-UE control-plane state manager inside the gNB-CU-CP that this thesis reimplements and measures.

2.2 Latency, throughput, and predictability in real-time software

Latency-critical systems are judged less by how much work they do on average than by how *predictably* they do it. Three metrics must be distinguished. *Throughput* is the sustained rate of completed operations; *latency* is the time an individual operation takes; and *jitter* is the variability of that latency. Because a system can have excellent mean latency and still miss deadlines often, the meaningful figures are the high-percentile *tail* latencies, the 99th (p99) and 99.9th (p99.9) percentiles, rather than the mean. At the scale of a network, tail latency dominates user-visible behaviour: when a single user action fans out into many internal operations, the slowest of them sets the response time, so rare per-operation delays become common per-request delays [18]. Modern data-centre and networking software increasingly operates in a regime where the relevant latencies are measured in microseconds, a range in which conventional operating-system and software overheads are no longer negligible [12]. Fine-grained measurement studies show that, even at moderate utilisation, hardware, operating-system, and application-level effects such as background interference, scheduling, interrupt routing, and power-saving states inflate the tail far beyond what queueing models predict [36].

Real-time systems are conventionally graded by the consequence of a missed deadline: in *hard* real-time a miss is a failure, in *firm* real-time a late result is useless but not catastrophic, and in *soft* real-time a miss merely degrades quality. A RAN control plane is best seen as soft-to-firm real-time: an occasional late signaling response degrades service rather than endangering it, but at fleet scale and under signaling storms

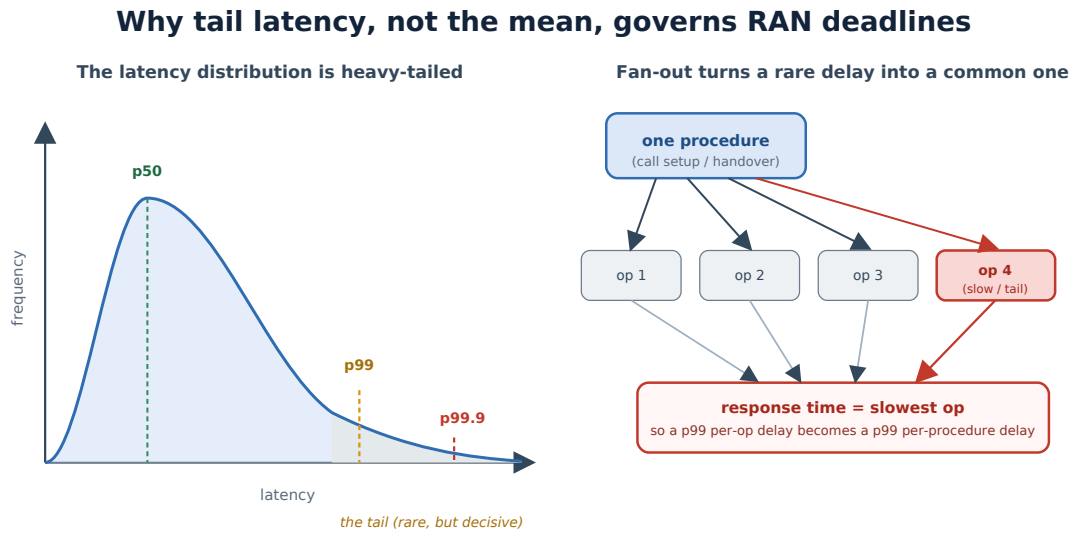


Figure 2.2: Why the tail, not the mean, governs RAN deadlines. The latency distribution is heavy-tailed (left), so the p99/p99.9 percentiles lie far from the median. Because a single procedure fans out into many dependent operations whose slowest member gates completion (right), a delay that is rare per operation becomes common per procedure; at fleet scale a one-in-a-thousand tail event is routine somewhere at every instant.

such degradations accumulate into a network-wide reliability problem. This is compounded by queueing behaviour: as offered load approaches service capacity, queueing delay grows sharply and the latency tail explodes near saturation. Characterising a system therefore requires driving it not just at nominal load but *into overload*, which is why the evaluation in this thesis sweeps load up to and beyond the saturation point.

Measuring tail latency correctly is itself a pitfall. A naive closed-loop load generator that waits for each response before issuing the next request suffers *coordinated omission*: when the system stalls, the generator stalls with it and simply fails to record the many requests that would have arrived during the stall, systematically hiding the very tail it is meant to expose. Sound measurement therefore requires an open-loop model with an intended schedule, correction for coordinated omission, high-dynamic-range latency histograms that retain resolution across many orders of magnitude, and statistically rigorous reporting with repeated runs and confidence intervals rather than single point estimates [25]. These techniques are applied in Chapter 7.

2.3 C++ for latency-critical systems

C++ is the default language for RAN control-plane software for good reasons. It offers zero-overhead abstractions, direct control over memory layout and allocation, deter-

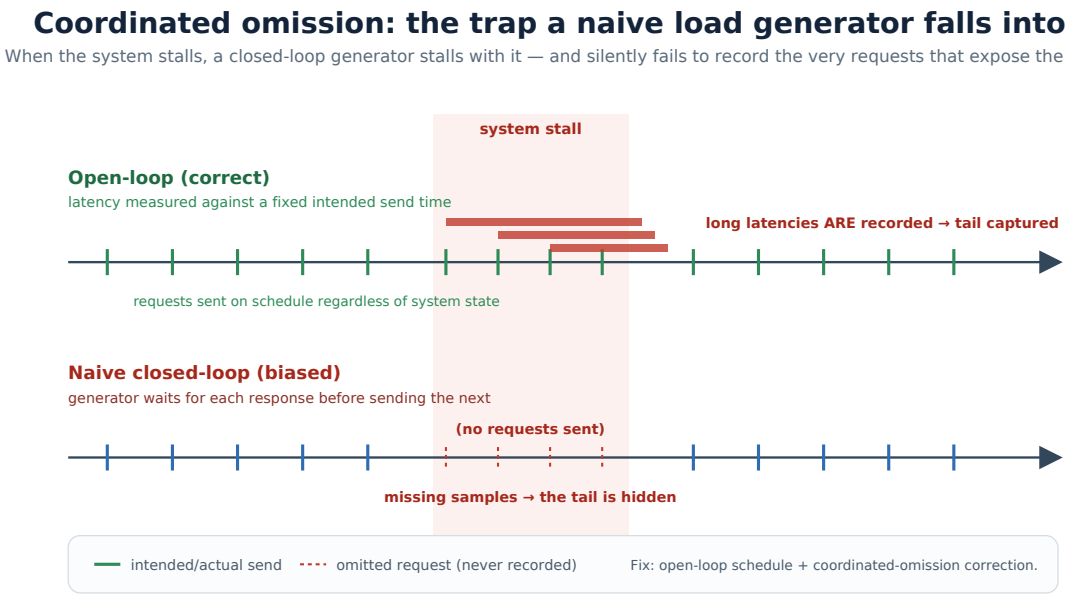


Figure 2.3: Coordinated omission. An open-loop generator (top) sends on a fixed intended schedule regardless of system state, so requests issued during a stall are recorded with their full latency and the tail is captured. A naive closed-loop generator (bottom) waits for each response before sending the next, so when the system stalls it stalls too and never issues the delayed requests, silently omitting exactly the samples that would expose the tail.

ministic destruction through the Resource Acquisition Is Initialization (RAII) idiom, and mature compilers, libraries, and profiling tools, all without a garbage collector that could inject unpredictable pauses. Modern C++ (C++11 and later) adds smart pointers (`std::unique_ptr`, `std::shared_ptr`) that automate ownership, move semantics that avoid needless copies, and a formal memory model with `std::atomic` for lock-based and lock-free concurrency. These features make it possible to write fast and, with discipline, reasonably safe systems code.

The difficulty is that safety in C++ remains the programmer's responsibility rather than a property the compiler enforces. RAII and smart pointers reduce but do not eliminate temporal errors: raw pointers and references can outlive the objects they point to, iterators can be invalidated by container mutation, objects can be used after a move, and nothing in the type system prevents a mutable object from being aliased and mutated concurrently from several threads. The result is a well-known catalogue of defects (use-after-free, double-free, buffer overflow, data races, deadlocks, and iterator invalidation) that are easy to introduce and expensive to find at telecom scale. Memory-corruption errors of this kind have been a persistent security problem for decades, and deployed mitigations remain only partial defences [56]. Their prevalence is well documented in industry: memory-safety issues account for roughly 70 % of the security vulnerabilities reported at Microsoft [41], and Google reports a sharp drop

in such vulnerabilities as memory-safe languages are adopted in Android [59]. This specific catalogue is developed in detail in Chapter 3 and forms the basis for the fault-injection study in Chapter 7.

C++ practice mitigates these defects with tooling rather than with language guarantees, and the tooling is inherently partial. Dynamic sanitizers such as AddressSanitizer (ASan), ThreadSanitizer (TSan), and UndefinedBehaviorSanitizer detect many errors, but only on the code paths that a given test actually executes with the right inputs, and they impose runtime and memory overheads that make them unsuitable for production; TSan in particular reports only those data races that happen to occur during a run, so a race that manifests only under a rare interleaving can pass unnoticed. Static analysers help but suffer false negatives and cannot reason about the whole program. Detection is therefore best-effort and after the fact, which is precisely the situation a language with compile-time guarantees is meant to change.

2.4 The Rust ownership and safety model

Rust offers memory safety and data-race freedom enforced at compile time through its ownership and borrowing model [33], together with a strong concurrency story and modern tooling (Cargo, integrated testing). It is the first industry-supported language to overcome the traditional trade-off between the safety of high-level languages and the low-level control of C and C++, achieving this with a type system based on ownership and borrowing that statically forbids the mutation of shared state, while confining the unavoidable exceptions to `unsafe` code encapsulated behind safe APIs [30]. The soundness of the safe fragment of the language, including the role of `unsafe` as an explicitly delimited escape hatch, has been established formally by the RustBelt project [29], and the aliasing discipline that even `unsafe` code must respect has been given a precise operational semantics [28]. Empirical evidence supports the model in practice: `unsafe` is used sparingly and in localised ways in real codebases [9, 51], and a study of all Rust memory-safety CVEs found that every one of them involved `unsafe` code, confirming that safe Rust keeps its promise [61]. The caveat is that unsafety propagates through dependencies: although fewer than a third of libraries use `unsafe` directly, a much larger fraction cannot be fully statically checked because `unsafe` hides somewhere in their call chain [22], which is a further argument for keeping the safety-critical core of a subsystem `unsafe`-free, as this thesis does. Crucially for a latency-critical setting, this safety is enforced by the type system and borrow checker rather than by runtime instrumentation, and is intended to be *zero-cost*: systems research argues Rust attains safety without the overhead of garbage collection or runtime checks, and even enables efficiency techniques that are impractical in C [11]. Whether that zero-cost claim holds

for a telecom control-plane workload is precisely the property this thesis evaluates.

Concretely, the borrow checker rejects at compile time the very use-after-free sketched in the introduction. The following program does not build, because `ctx` is moved (dropped) while a reference to it is still live:

```
let ctx = UeContext::new(42);
let r = &ctx;      // immutable borrow of ctx
drop(ctx);         // move/free ctx ...
use_ctx(r);        // error[E0505]: cannot move out of ‘ctx‘
                   //               because it is borrowed
```

The same discipline extends to concurrency. Rust encodes thread-safety in the type system through the `Send` and `Sync` marker traits: a value may cross a thread boundary only if its type is `Send`, and may be shared by reference across threads only if it is `Sync`. Sharing a value that permits unsynchronised mutation is therefore a compile error, not a latent race; this is the property Rust markets as "fearless concurrency." Error handling is likewise explicit: fallible operations return a `Result` value that the caller must handle, rather than raising an exception that unwinds through hidden control flow. Both the cost and the control flow of the failure path therefore stay visible, which is an advantage in latency-sensitive code where surprises in the tail are what matter.

2.5 The systems-language landscape

Rust is not the only memory-safe language, but it occupies a distinctive position for this problem. Managed languages such as Java, C#, and Go achieve memory safety through a runtime and garbage collection. That is an excellent trade for much software, but it is a poor fit for a tail-latency-critical control plane: garbage collection introduces pauses and timing variability that are a recognised source of tail latency in large systems [18], and a managed runtime also cedes the low-level control over memory layout and scheduling that this domain relies on. At the other extreme, C is as memory-unsafe as C++ while offering fewer abstractions for building large systems. Rust is the first widely supported language to combine memory and thread safety, enforced at compile time, with the absence of a garbage collector and performance intended to be competitive with C++ [30, 11]; cross-language benchmark surveys place Rust's throughput and memory use alongside C and C++ while it retains its safety guarantees [13]. That combination is exactly why the meaningful comparison for RAN control-plane software is C++ against Rust, and why managed languages are out of scope here.

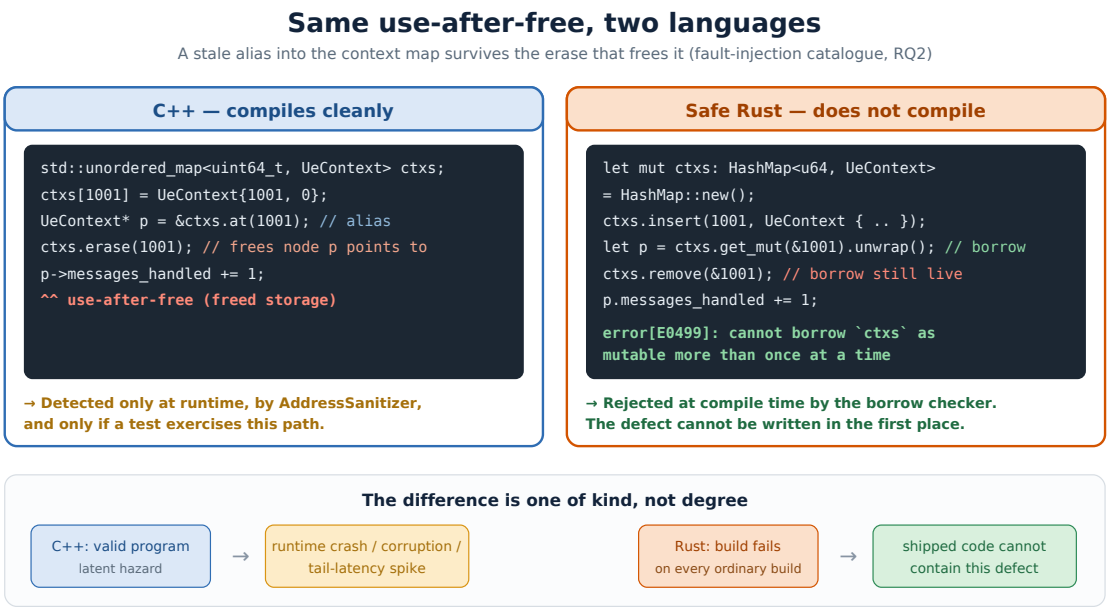


Figure 2.4: The same use-after-free defect in C++ and in safe Rust. The stale alias into the context map survives the `erase`/`remove` that frees it: in C++ this compiles and is caught only at runtime (and only under AddressSanitizer, on a path a test exercises), whereas in safe Rust the borrow checker rejects it at compile time (`E0499`), so the defect cannot be written at all. The difference is one of kind, not degree.

2.6 Concurrency models for control-plane software

How concurrency is structured shapes both correctness and tail latency. The dominant model is *shared-memory* concurrency, in which threads mutate common state guarded by mutexes and atomics; it is flexible but is the source of races, deadlocks, lock contention, and priority inversion, and its worst-case timing is hard to reason about. The alternative is *message-passing* or *shared-nothing* concurrency, in which state is partitioned so that each piece is owned by a single thread and all interaction happens through messages, as in the actor model and in sharded designs. Shared-nothing trades some flexibility for scalability and far simpler reasoning, and it maps naturally onto per-UE partitioning in a control plane, where each UE's state is largely independent. The choice is not neutral with respect to the quantity this thesis measures: thread-per-core structuring has been shown to change application tail latency in its own right [21], which is one reason the model is held fixed across both implementations rather than left to each language's idiom.

A second, orthogonal choice is how work is queued between producers and consumers. Unbounded queues mask overload and let latency and memory grow without limit; *bounded* queues with either blocking (backpressure) or shedding bound worst-

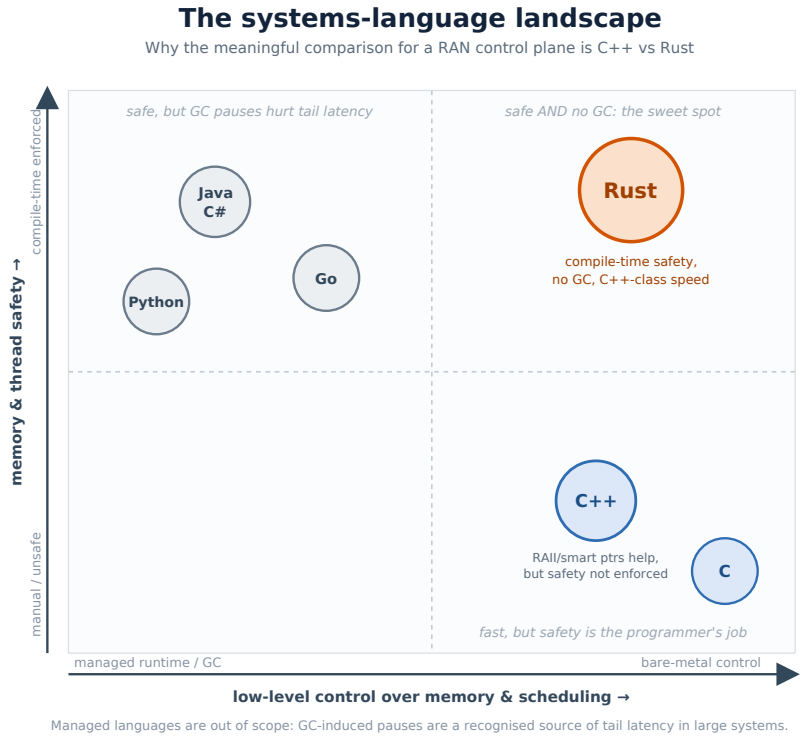


Figure 2.5: The systems-language landscape along two axes: low-level control over memory and scheduling (horizontal) and memory/thread safety (vertical). Managed languages (Java, C#, Go, Python) are safe but pay a garbage-collection cost that hurts tail latency; C and C++ give bare-metal control but leave safety to the programmer. Rust occupies the sweet spot of compile-time safety with no garbage collector, which is why the meaningful comparison for a RAN control plane is C++ against Rust.

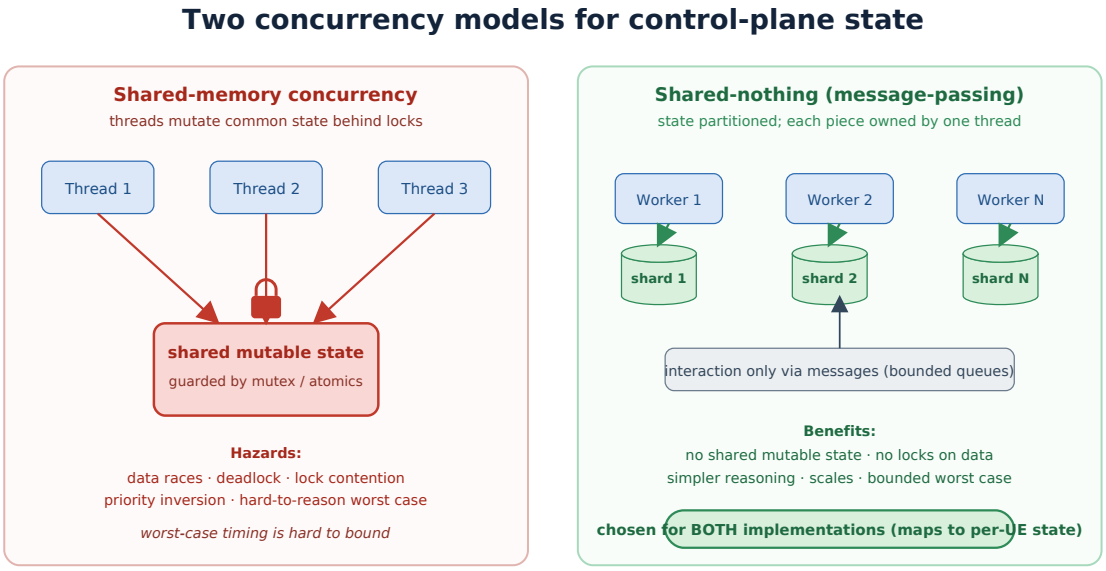


Figure 2.6: Two concurrency models for control-plane state. Shared-memory concurrency (left) lets threads mutate common state behind locks and is the source of data races, deadlock, and lock contention, with a worst case that is hard to bound. Shared-nothing message-passing (right) partitions state so each shard is owned by a single worker and all interaction happens through bounded queues; this eliminates locks on context data and maps naturally onto per-UE partitioning, and is the model adopted for both implementations.

case memory and make overload visible, which is essential for predictable tail latency. Lock-free structures can reduce contention but are notoriously difficult to implement correctly. These considerations directly motivate the architecture adopted for both implementations in Chapter 6: a shared-nothing, thread-per-shard design with bounded multi-producer/single-consumer queues and blocking backpressure, chosen so that any measured latency difference reflects the language rather than a divergent concurrency strategy.

2.7 Interoperability between C++ and Rust

Adopting a new language in a large existing system is necessarily incremental: a Rust component must coexist with, and be callable from, a substantial C++ codebase. The common denominator for such interoperability is the C application binary interface (ABI). Rust can expose functions with `extern "C"` linkage and hand out *opaque handles* to Rust-owned objects, which C++ then calls through a C header; tooling automates parts of this, including `bindgen` (generating Rust bindings from C headers), `cbindgen` (generating C headers from Rust), and the `cxx` crate (a safer bidirectional bridge). Interoperability is not free: crossing the boundary steps outside the checked

language, so the seam must contain `unsafe` code; passing anything beyond simple values requires marshalling; and the shape of the foreign interface has to be designed deliberately. The size of that seam, how much `unsafe` it concentrates, and the per-call crossing cost are exactly the quantities measured for RQ4 in Chapter 7.

A distinction has to be drawn at this point, because it bounds what any of those measurements can establish. Concentrating `unsafe` in the seam is a statement about what the compiler checks, not about what can happen at run time. Once a Rust component is linked into a C++ host they share one address space, and a defect on the C++ side can corrupt Rust-owned data irrespective of how much of the Rust side the borrow checker verified; this has been demonstrated experimentally, and the proposed remedies rely on hardware or runtime isolation of the two heaps rather than on the discipline of the interface [53, 32]. The defect classes arising specifically at Rust/C boundaries have been catalogued separately from those of either language [38], and the binding generators themselves are known to carry bugs [14]. The consequence for this thesis is stated once here and honoured throughout: an `unsafe`-free core means the core's own defects are ruled out at compile time, and it does not mean the core is protected from its host.

2.8 Research gap

The preceding sections establish three things. First, the RAN control plane is a soft-to-firm real-time, highly concurrent, latency-critical domain in which tail behaviour under overload, not average throughput, governs the quality experienced by the network. Second, C++ supplies the control and performance this domain demands but leaves whole classes of memory and concurrency defects to best-effort, after-the-fact detection. Third, Rust promises to prevent those classes at compile time at no runtime cost, with a credible incremental adoption path through C-ABI interoperability.

The domain is not entirely unstudied. Two vendor-hosted master's theses have compared candidate languages for RAN and 5G control-plane software, one weighing runtime cost against development efficiency across C++, Rust, and Go [31], the other measuring median and 99th-percentile latency for control-plane procedures across five asynchronous programming frameworks [8]. Both are directly relevant, and the second reports a tail difference unfavourable to Rust. Neither, however, isolates the language: in both, the language varies together with the runtime, the concurrency model, and the implementation, so a measured difference cannot be attributed to the language rather than to the framework built on it.

What is therefore missing from the literature is *direct, controlled* evidence for a representative control-plane subsystem: whether Rust actually holds its tail latency

and predictability under overload against modern C++ *when the concurrency model, algorithms, and data structures are held fixed*, exactly which defect classes it eliminates and at what stage the remainder are caught, what its safety model costs in throughput and resources, and what incremental adoption costs in practice. Closing this gap is the purpose of the four research questions. The next chapter grounds them in the specific subsystem, defect classes, and timing properties at stake; Chapter 4 turns them into objectives and testable hypotheses; and Chapter 5 sets out the methodology by which they are pursued.

3. Problem Context: UE Context Management and Predictability

3.1 The UE Context Management subsystem

The subsystem under study is a UE Context Management service, modelled on a real Nokia gNB-CU-CP service domain that owns the UE context lifecycle, RRC procedure execution [4], and mobility/handover. Those are the very functions the 3GPP architecture assigns to the control-plane part of a disaggregated gNB: the CU-CP hosts the RRC and the control-plane part of PDCP and manages the UE context, delegating user-plane bearer handling to the CU-UP over the E1 interface [1, 2, 3]. It is a good representative because it is core RAN L3 functionality, performance- and latency-sensitive, inherently stateful and concurrent, and self-contained enough to specify precisely and benchmark fairly. The modelled functionality includes UE context creation and deletion, state transitions (e.g. idle to connected), signaling message handling, and concurrent context lookup under load. Figure 3.1 shows the UE-context lifecycle state machine that both implementations encode identically; illegal transitions (for example signaling an `Idle` context) return a typed error rather than mutating state, and this state machine is exactly what the mixed workload of Section 7.2 walks each UE through.

Representativeness and tractability. Two properties make UECM a good study object for a language comparison. It is *representative*: per-UE mutable state, a strict operation lifecycle, high inter-UE concurrency, and the hard latency and reliability targets the 5G system places on control-plane procedures [5] are exactly the properties that make production RAN control-plane code difficult, so conclusions about how a language handles them transfer to the broader class of stateful, concurrent control-plane services. It is also *tractable*: the subsystem is small enough to specify precisely, implement faithfully in two languages, and reason about in full, which is what makes a controlled, parity-tested comparison possible at all. A larger or less well-bounded subsystem would improve external validity at the cost of the internal validity on which

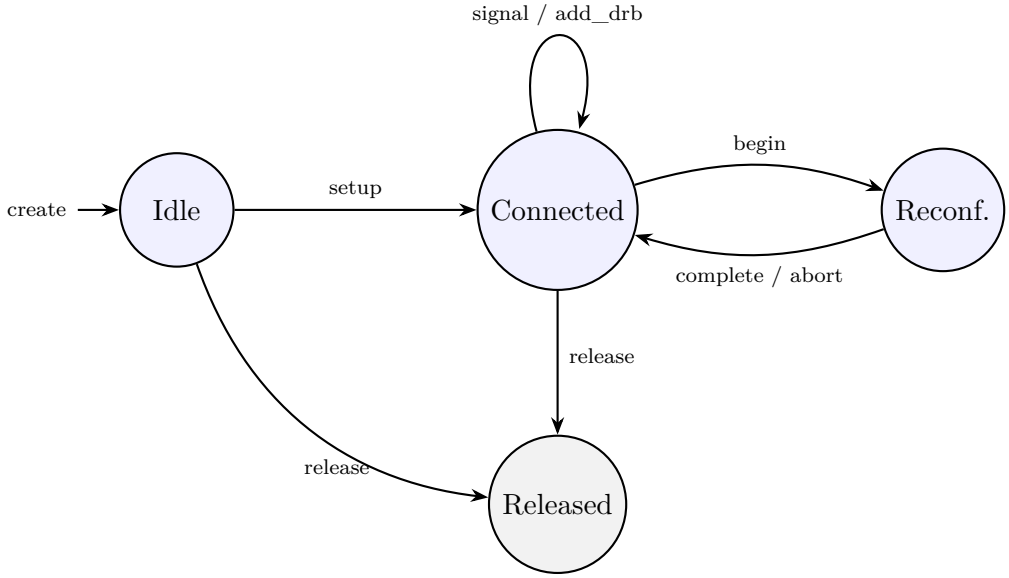


Figure 3.1: UE-context lifecycle state machine. `create` allocates a context in `Idle`; `setup` connects it; `signal` and `add_drb` are handled in place while `Connected`; a reconfiguration/handover moves through `Reconfiguring` and back on `complete` or `abort`; `release` tears the context down. Operations invalid for the current state return an `OpError` instead of changing state.

a fair language comparison depends; UECM is chosen as the point that keeps both within reach.

Operation set. The interface exercised by the study is deliberately small and lifecycle-oriented: create and release a context; set up a connection (`Idle`→`Connected`); handle an in-session signaling message; add a data radio bearer; begin, complete, or abort a reconfiguration/handover; and look up a context's state under concurrent load. Each operation is total with respect to the state machine of Figure 3.1: an operation that is illegal in the current state returns a typed error and leaves the context unchanged, rather than throwing or corrupting state. This closed, well-typed operation set is what lets the same test vectors and the same workload drive both implementations along identical paths (Section 6.7).

3.2 Memory-safety and concurrency defect classes

The defects that afflict C++ control-plane code of this kind fall into two families. The *memory-safety* family concerns the lifetime and bounds of objects: a *use-after-free* accesses storage that has already been released, a *double-free* releases the same storage twice, and a *buffer overflow* reads or writes past the end of an allocation. The *concurrency* family concerns unsynchronised or ill-ordered access from multiple threads: a *data race* is a pair of unsynchronised accesses to the same location of which

at least one is a write, a *deadlock* is a cyclic wait for locks, and *iterator invalidation* is the use of an iterator or reference into a container after a mutation has moved or freed the underlying storage. What these share is that they are all *silent* in C++: each is accepted by the compiler and, on the paths ordinary tests happen to exercise, may appear to work, surfacing only later as corruption, a crash, or an unpredictable latency spike. These are the same memory-corruption classes that decades of security research have struggled to contain in C and C++ [56].

Each class has a natural incarnation in UE Context Management, which is why the subsystem is a good vehicle for studying them:

- **Use-after-free.** A signaling message holds a raw pointer or reference to a UE context that a concurrent `release` has already erased from the context map; the in-flight handler then dereferences freed storage.
- **Double-free.** A release path runs twice for the same UE, for example because a retransmitted release is not made idempotent, freeing the context's owned resources a second time.
- **Iterator invalidation.** Code iterates a context's data-radio-bearer list while adding or removing a bearer, so the iterator refers into storage the container has since reallocated.
- **Data race.** Two workers update the same context field, an activity counter or a state flag, without synchronisation, so the accesses race.
- **Unguarded shared mutation.** A context is shared by reference across threads and mutated without a lock or other discipline, a race whose root cause is the absence of any ownership rule for the shared state.

These five classes lie squarely within the guarantees Rust claims to provide, which makes them the natural test of whether that claim holds. Testing only them, however, would build the expected answer into the choice of test cases, so two further classes are carried into the study for the opposite reason, because there is good reason to expect that safe Rust does *not* prevent them:

- **Lock-order deadlock.** A handover locks the source UE context and then the target while a concurrent handover in the opposite direction locks target then source, and the two wait for each other indefinitely. Nothing about object lifetimes or aliasing is violated; the fault is in the order of acquisition.
- **Lost update.** Two workers increment a context's activity counter through a read-modify-write split across two separate critical sections. Every individual

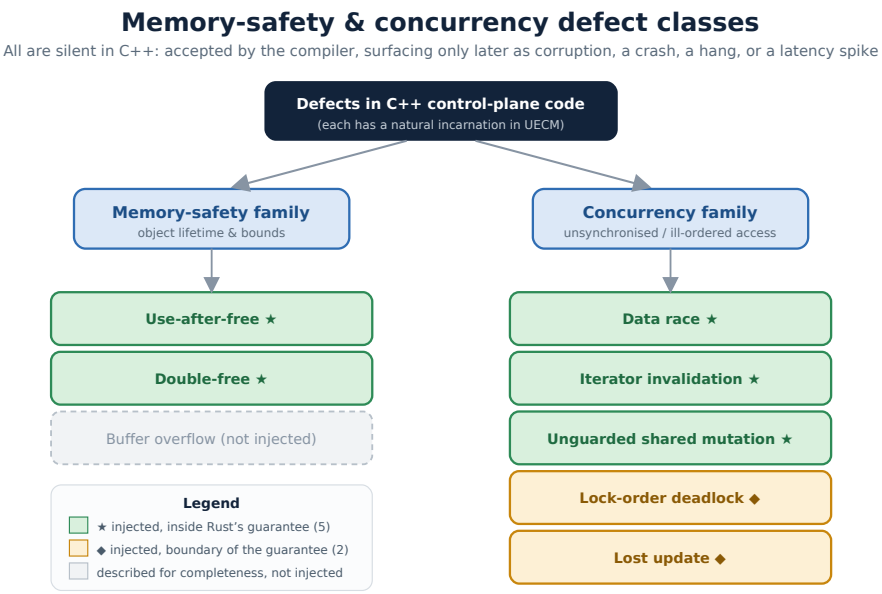


Figure 3.2: The defect classes that afflict C++ control-plane code, split into a memory-safety family (object lifetime and bounds) and a concurrency family (unsynchronised or ill-ordered access). Starred classes are the five injected from inside Rust's guarantee; the two diamond-marked classes, lock-order deadlock and lost update, are injected precisely because they lie outside it, so the catalogue locates the boundary of the guarantee rather than only confirming it. Buffer overflow is described for completeness but is not injected. All are silent in C++, surfacing only later as corruption, a crash, a hang, or a latency spike.

access is correctly synchronised, so this is a *race condition* rather than a *data race*: the atomicity violation lives in the composition of the operations, not in the memory accesses themselves, and it corrupts a value silently instead of crashing.

Together these seven form the catalogue injected in Chapter 7, and the split between the two groups is what lets the study report a boundary rather than a verdict. Buffer overflow is described above for completeness but is not injected, because neither implementation performs raw pointer arithmetic on the context store, so an injected instance would be an artificial construct rather than a plausible bug in this subsystem.

3.3 Tail latency and predictability in RAN

Section 2.2 set out why the tail rather than the mean governs a latency-critical system, and what measuring it correctly demands. What that section left general, this one makes specific, because two properties of the RAN setting are what turn a methodological preference into the governing metric.

The first is *fan-out*: a single user-visible action, a call setup or a handover, decomposes into many dependent control-plane operations, and because the slowest

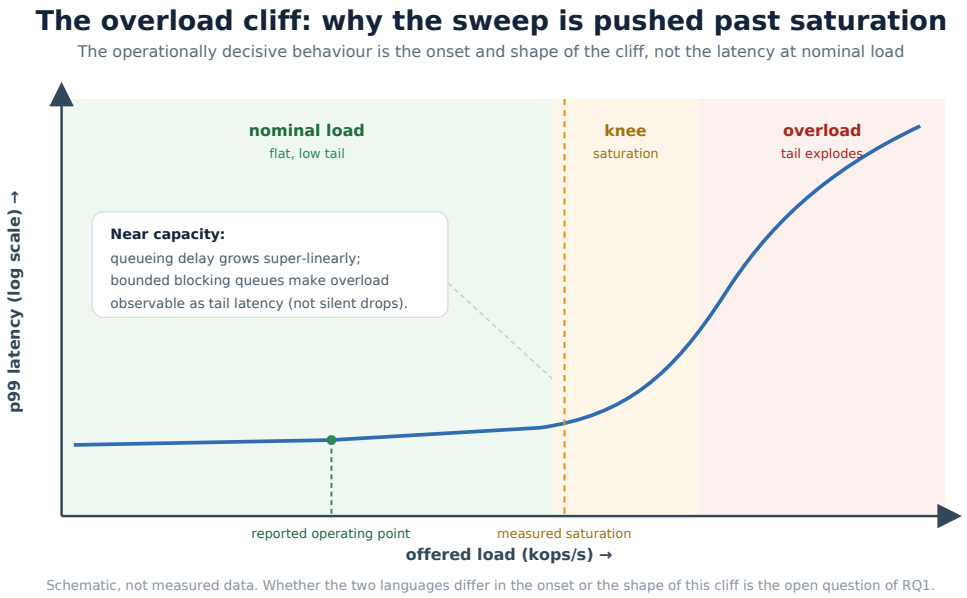


Figure 3.3: The overload cliff. Tail latency stays low and flat while offered load is below service capacity, then climbs steeply once the system is driven past saturation and requests queue behind the bounded, blocking queues. The operationally decisive behaviour is the onset and shape of this cliff, which is why the load sweep is pushed beyond the saturation point rather than stopping at nominal load. The figure is a schematic of the phenomenon and not a plot of this study's data: whether the two languages differ in where the cliff begins or how steeply it rises is exactly what RQ1 asks, and Section 7.5.1 reports that the development host cannot resolve it.

of them gates completion, a latency that is rare per operation becomes common per procedure [18]. The second is *scale*: a busy site drives thousands of such procedures per second across millions of connected devices, so a tail event occurring once in a thousand operations is not an edge case but a routine occurrence somewhere in the fleet at every instant. Taken together, the p99.9 of a single UE Context Management operation is not a curiosity at the margin of a distribution; it is a rate of degraded procedures the network sustains continuously.

The study therefore characterises the full latency distribution, focusing on the high percentiles (p99, p99.9, p99.99), the maximum, and jitter, and on how these degrade as offered load approaches and then exceeds saturation (Chapter 7). Stopping at nominal load would miss the regime that decides service quality, since queueing delay grows super-linearly near capacity; it is also the regime in which a difference between two implementations of one design, if there is one, has room to become visible.

4. Objectives

4.1 Deficiencies in existing comparisons

The gap argued in Section 2.8 is only as good as the search that looked for it, so the search is stated rather than assumed. Candidate literature was gathered from the ACM Digital Library, IEEE Xplore, USENIX, and Google Scholar using combinations of the terms *Rust*, *C++*, *memory safety*, *tail latency*, *systems programming*, *performance comparison*, and *RAN / 5G control plane*, restricted to peer-reviewed venues except where an industry or government report is itself the object of study (the memory-safety mandates and vendor vulnerability statistics). The result set was then extended by backward and forward citation chaining from the core Rust-safety papers, and filtered to work that either measures a systems language empirically, formalises Rust's guarantees, or reports adoption experience. The domain literature (3GPP specifications and the O-RAN survey) was added separately to fix the architectural context. The search returned no study that measures the tail-latency behaviour of a Rust and a C++ implementation of the same stateful, concurrent control-plane subsystem under overload.

What it did return is the baseline this thesis works against, and two results deserve naming because they are the strongest evidence that the answer may simply be "no difference". Cross-language studies report that compiled systems languages achieve broadly similar time and energy efficiency, well ahead of managed and interpreted ones [47]. Closer to this domain, an idiomatic Rust 10 Gbit/s user-space network driver executes many more instructions per packet than its C reference yet runs only about 4% slower [19], which suggests Rust's safety checks are largely absorbed by out-of-order execution rather than being literally free. Both are average-case results on general-purpose or packet-forwarding workloads, so neither addresses the four questions Section 2.8 left open. They establish that this comparison is worth making under controlled conditions, not that it has been made.

4.2 Solution objectives

The overarching objective is a fair, reproducible comparison of modern C++ and Rust for a RAN control-plane subsystem, together with reusable artifacts and evidence-based guidance. It decomposes into concrete objectives, each aligned with a research question so that progress is traceable:

O1 (predictability, RQ1) Quantify and compare the latency distributions of the two implementations, focusing on the p99/p99.9 tail and jitter, across a load sweep that reaches overload, and identify where along that sweep, if anywhere, the languages diverge.

O2 (defect prevention, RQ2) Determine, for a catalogue of defect classes representative of the subsystem, which are prevented at compile time in safe Rust and, for those that are not, at what stage and cost they are detected in C++.

O3 (overhead, RQ3) Measure the resource cost, throughput, CPU utilisation, and memory behaviour, of each implementation under matched conditions, isolating any systematic penalty attributable to Rust's safety model.

O4 (migration, RQ4) Build and measure a realistic incremental-adoption prototype, quantifying the size of the interoperability seam, the scope of `unsafe` it requires, and the per-call cost of crossing the language boundary.

O5 (artifacts and guidance) Deliver a frozen specification, a parity-tested pair of implementations, a reproducible benchmarking framework, a defect-class taxonomy, and engineering guidelines that practitioners can reuse for language-selection decisions in cloud-native RAN evolution.

4.3 Hypotheses

The objectives are approached with three prior expectations, stated so that the evaluation can confirm or refute them rather than merely describe results after the fact. **H1:** for this workload the two languages are performance-equivalent in the operationally relevant region below saturation, with any difference confined to the far tail. **H2:** safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ leaves to best-effort, after-the-fact detection. **H3:** the run-time cost of C++-to-Rust interoperability is negligible relative to the cost of the operations invoked, so that the true cost of incremental adoption is engineering effort rather than performance. These hypotheses are revisited against the evidence in Chapter 8.

4.4 Success criteria

The study is successful to the extent that it produces *decision-grade* evidence, evidence on which a practitioner could base a language choice, rather than merely a set of measurements. Concretely this requires: that any performance comparison be reported with repeats and confidence intervals so that a claimed difference is distinguishable from run-to-run noise; that the defect-prevention result be demonstrated on concrete, compilable programs rather than argued from language semantics alone; that the migration cost be read off a working prototype rather than estimated; and that the whole be reproducible from published artifacts and a public specification. Meeting these criteria is what distinguishes the contribution from the general-purpose, average-case comparisons whose deficiencies motivate the work.

4.5 Scope and boundaries

The work focuses on a single representative subsystem studied under controlled, synthetic conditions, and its boundaries are drawn deliberately to protect the internal validity of the language comparison. In scope are the UE Context Management functionality of Chapter 3, the shared-nothing concurrency design, the load sweep to overload, the fault-injection catalogue, and the FFI prototype. Explicitly out of scope are the full gNB system and its remaining control-plane domains, PHY/L2 processing and the data-heavy user plane, real captured traffic traces (as opposed to a spec-justified synthetic workload), managed-language alternatives (excluded on the tail-latency grounds argued in Chapter 2), and production deployment and operational concerns such as upgrade, observability, and multi-tenancy. These exclusions trade external validity for the controlled conditions a fair language comparison needs, and each is revisited as a limitation in Chapter 8.

5. Research Methodology

The previous chapter set out what the study must achieve; this one sets out how. It states the overall research strategy, the method chosen for each research question and why it is appropriate, the controls that make a language comparison fair, the statistical treatment of the measurements, and the reproducibility and ethical commitments under which the work is carried out.

5.1 Research approach

The thesis follows an empirical, experimental research approach with a constructive (design-science) element. A software artifact, two functionally equivalent implementations of the subsystem together with a benchmarking framework, is built against a single frozen specification and then evaluated under controlled experiments. Evidence is triangulated from three complementary methods: quantitative performance measurement under a swept load (addressing RQ1 and RQ3), a controlled fault-injection study of defect detection (RQ2), and a qualitative-plus-quantitative FFI migration case study (RQ4). Fairness is treated as a first-class concern: because the goal is to isolate language effects, the study fixes the specification, algorithms, concurrency model, allocator, and build settings across both implementations, and reports variance over repeated runs.

Table 5.1 maps each research question to the method, artifact, data, and metric used to answer it, making the chain from question to evidence explicit and traceable.

5.2 A constructive, comparative experiment

The four research questions are causal and comparative: each asks how an outcome (tail latency, prevented defects, resource use, or migration cost) depends on a single independent variable, the implementation language. Answering such questions credibly requires that the language be the *only* systematic difference between the two objects under test. This rules out the most common form of language comparison in the lit-

Table 5.1: Research design: how each research question is answered. Every question is tied to a concrete method, a published artifact, the data it produces, and the metric on which the conclusion rests.

RQ	Method	Artifact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness; latency histograms	p99/p99.9 tail latency and jitter vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak RSS, CPU utilisation, throughput
RQ2	Controlled fault injection	Defect catalogue; ASan/TSan and `rustc` logs	defect-class taxonomy (compile-time / runtime / undetected)
RQ4	FFI migration prototype	C++ host + Rust C-ABI seam	seam size, `unsafe` scope, per-call interop cost

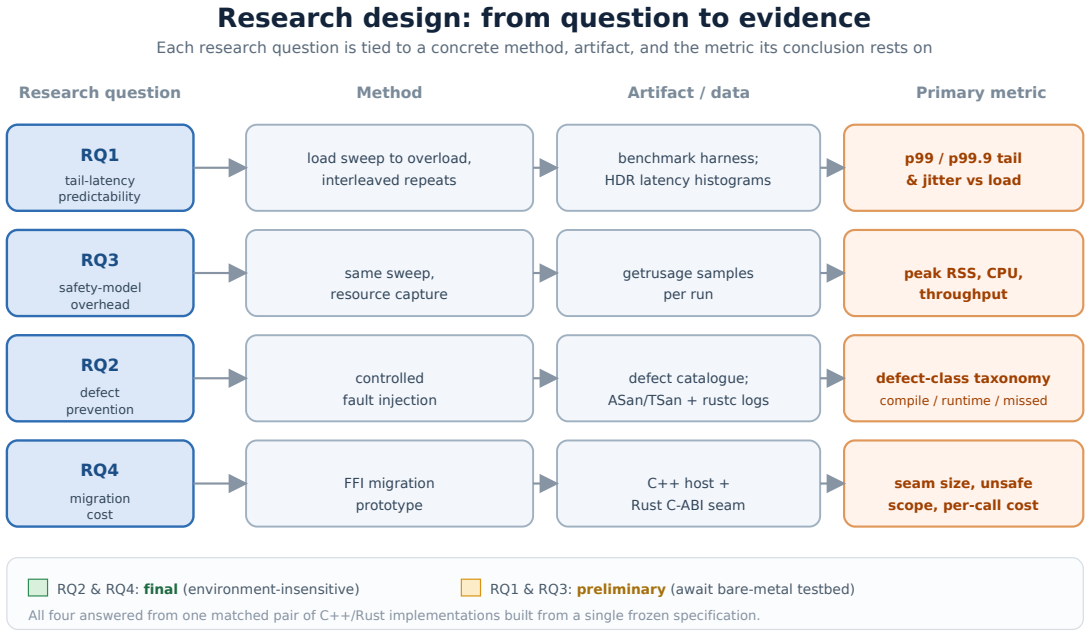


Figure 5.1: The research design as a flow, showing for each research question the method, the artifact and data it produces, and the primary metric its conclusion rests on. All four questions are answered from one matched pair of C++/Rust implementations built from a single frozen specification; RQ2 and RQ4 are environment-insensitive and reported as final, while RQ1 and RQ3 are preliminary until the bare-metal testbed campaign.

erature, in which two independently written programs, or two unrelated projects, are measured and their differences attributed to the language; such comparisons confound the language with the design, the libraries, and the intent and skill of their authors. The thesis therefore adopts a constructive (design-science) strategy: rather than locating two pre-existing systems, it *builds* the object of study as a matched pair of implementations derived from one frozen specification, so that the comparison is between two encodings of the same design rather than between two designs. The artifact, the specification, the two implementations, and the benchmarking harness, is thus simultaneously the contribution of the thesis and the instrument through which it is measured.

The experiment is empirical and quantitative where the questions admit numbers (RQ1, RQ3, and the per-call cost of RQ4) and systematic-but-qualitative where they do not (the kind-of-guarantee distinction in RQ2 and the API-design effort in RQ4). Evidence is deliberately triangulated across three complementary methods rather than resting on a single campaign, so that a weakness in any one method, for example the environment sensitivity of tail-latency measurement, does not undermine the conclusions as a whole.

5.3 Methods for the four research questions

Table 5.1 pairs each question with a method; this section states what each method does and why it is appropriate.

5.3.1 Performance measurement under a load sweep (RQ1, RQ3)

RQ1 and RQ3 are answered by driving both implementations with one deterministic synthetic workload across a sweep of offered load, from light load through saturation and into overload, and recording latency and resource use at each level. Two design decisions make the measurement sound. First, the load generator follows an *open-loop* schedule with a fixed intended send time per request, and latency is measured against that intended time rather than against the moment the request is actually admitted; this is what corrects for coordinated omission, the bias by which a naive closed-loop generator stalls together with the system and silently fails to record the very requests that would have exposed the tail (Chapter 2). Second, the sweep is pushed past saturation deliberately: the operationally decisive behaviour of a control plane is not its latency at nominal load but the onset and shape of the overload cliff, and that is visible only once offered load exceeds service capacity. Latency is captured in

high-dynamic-range histograms configured identically on both sides, and resource use (peak resident set size, CPU utilisation, throughput) is sampled through the operating system, so that the same measurement apparatus is applied to both languages.

5.3.2 Controlled fault injection (RQ2)

RQ2 asks which defect classes a language prevents, and it cannot be answered by measuring defect-free code: the absence of a bug in a passing test says nothing about whether the language would have caught it. The method is therefore adversarial, following the established practice of software fault injection, in which representative faults are deliberately introduced to assess how a system behaves in their presence [44]. A catalogue of representative C++ defects, drawn from the classes that afflict this subsystem (Chapter 3), is written *deliberately*, each as a small self-contained program flavoured with the domain (a stale pointer into the context map, a double release, a mutation during iteration, an unsynchronised shared update). Each C++ defect is then built and run under AddressSanitizer and ThreadSanitizer, recording which tool, if any, detects it and at what stage; the analogous construct is written in safe Rust and compiled, recording the compiler's outcome and, where it compiles, the behaviour of the resulting program. Because the independent variable is again only the language while the injected fault is held fixed, the difference in outcome, compile-time rejection, runtime detection, or silent miss, is attributable to the language's guarantees rather than to the specific bug. The result is a taxonomy rather than a performance number: the method establishes the *kind* of protection each language offers.

One design decision determines whether that taxonomy can be trusted. A catalogue drawn only from classes the language is known to prevent would produce a foregone conclusion: the method would be capable of confirming the hypothesis but not of disturbing it, and the result would describe the author's choice of cases as much as the language. The catalogue is therefore constructed in two groups. The first contains classes that lie inside Rust's stated guarantee; the second contains classes deliberately chosen from just outside it, where ownership and the `Send/Sync` discipline give no protection. Each case carries a recorded expectation, prevented or not prevented, and the runner reports a violation in either direction, so an unexpected outcome would surface as a discrepancy rather than being absorbed into the narrative. What this buys is a boundary rather than a score: the study can state not only which defect classes the language removes but which comparable ones it leaves in place, and the second half of that statement is the part a practitioner planning a migration needs.

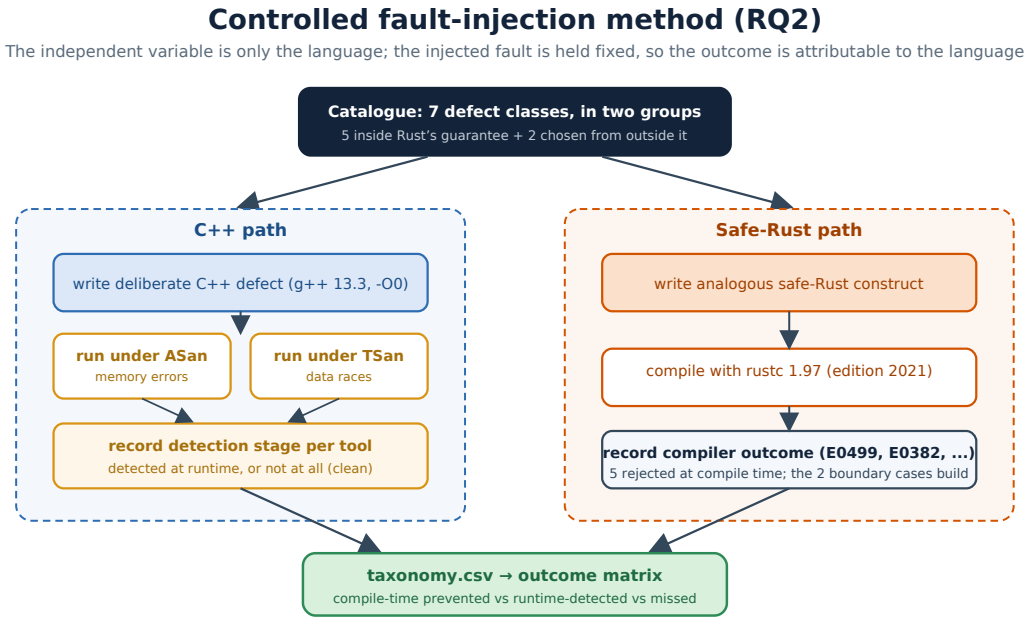


Figure 5.2: The controlled fault-injection method for RQ2. A fixed catalogue of representative defect classes is confronted in two languages: each C++ defect is built and run under AddressSanitizer and ThreadSanitizer to record the detection stage, while the analogous safe-Rust construct is compiled to record the compiler's outcome and, where it compiles, run to record its behaviour. Because only the language varies while the injected fault is held fixed, the difference in outcome is attributable to the language's guarantees. The catalogue spans classes inside and outside those guarantees so that the method locates their boundary rather than only confirming them.

5.3.3 Foreign-function-interface migration case study (RQ4)

RQ4 concerns the practical cost of adopting Rust inside an existing C++ system, which is an engineering question as much as a performance one. It is answered by constructing a working prototype in the realistic direction of migration: an existing C++ host calling a Rust component across a C application binary interface. Three quantities are then measured on it. The *size* of the interoperability seam counts its entry points, lines of code, and dependencies. The *scope* of `unsafe` records where the checked language is left and whether the safety-critical core is affected. The *per-call cost* is the price of crossing the boundary, measured against the same operation invoked natively so that the comparison isolates what the seam adds rather than what the operation costs. The first two are structural and are read off the code; the third is a matched pair of microbenchmarks. This combination of a structural account and a quantitative cost measurement is what lets the study separate the run-time cost of interoperability from its true cost, which lies in API design and human learning.

5.4 Fairness as a controlled-variable strategy

Because every research question attributes a difference to the language, the validity of the whole study depends on holding everything else equal, and fairness is treated as the study's central experimental control rather than as an afterthought. It is enforced at several levels. The *specification* is frozen before any code is written and fixes the public API, the state machine, the data structures and their algorithmic strategy, and the sharding and concurrency model (Section 6.1). The *run-time substrate* is equalised by linking the same memory allocator on both sides and matching build-optimisation levels as closely as the two toolchains allow. The *workload* is produced by a driver that is identical in structure across languages and, for a given seed and active-UE population, drives both implementations through exactly the same sequence of operations, so that any latency difference reflects how each language executes the same work rather than a difference in what work is done. Where the two languages genuinely cannot express the same construct, the deviation is not hidden but recorded in a forced-idiom log so it can be weighed explicitly in the discussion. What remains uncontrolled, principally single-author skill asymmetry, is acknowledged as a threat to validity (Chapter 8) rather than claimed to be eliminated.

5.5 Statistical treatment of measurements

Performance measurements are non-deterministic, so single-run numbers are avoided throughout. Each configuration (language × allocator × load level) is measured over many independent repeats, and the four variants are *interleaved* within each repeat so that slow drift, thermal effects, and background activity perturb them equally rather than biasing one. Results are summarised by the median with a nonparametric bootstrap 95% confidence interval rather than by the mean with a standard deviation, because latency distributions are heavy-tailed and skewed and a distribution-free interval on the median makes no normality assumption. This follows established guidance on statistically rigorous performance evaluation, which warns that reporting a single number invites conclusions that run-to-run variation does not support [25]. Tail percentiles (p99, p99.9, p99.99) are read from the HDR histograms, whose logarithmic bucketing retains three significant figures of resolution across the whole range so that high percentiles are not quantised away. Intervals are reported in full rather than as a symmetric half-width, because a bootstrap interval on the median of a heavy-tailed sample is markedly asymmetric and a half-width wide enough to cover its upper bound would place the lower bound below zero.

Interval estimates alone, however, are a weak basis for a comparison. Two intervals can overlap while one variant is ahead in every single repeat, and they can separate on a difference that does not reproduce. The study therefore also tests the comparison directly, and the interleaved design is what makes this possible: because the four variants are measured back to back inside each repeat, the runs form a matched design in which the interference common to a block cancels in the pairwise difference. Language is compared against language with the allocator held fixed, using an exact two-sided paired permutation test on the repeat-wise differences. The null hypothesis is that a pair differs only by run-to-run noise, under which the sign of each paired difference is exchangeable; for the repeat counts used here every one of the 2^n sign assignments is enumerated rather than sampled, so the resulting p-value is exact and, like the bootstrap, assumes nothing about the shape of the distribution. Because the same comparison is made for several metrics and both allocators, the family of tests is corrected with the Holm step-down procedure, which controls the family-wise error rate without assuming independence between the tests.

Two decisions about how that family is read are taken here rather than after the results, because either could otherwise be made to suit whatever the data turned out to be.

The first is which comparisons enter it. Achieved throughput is excluded. The representative operating point is chosen below saturation, where every variant com-

pletes work as fast as it is offered, so achieved throughput is pinned to the offered load by construction and cannot express a language effect whatever the languages do. Testing it would re-measure the operating point rather than the code. It would also not be free: Holm divides its budget across the whole family, so two comparisons that cannot resolve anything would make the correction harsher on the ones that can. Throughput is instead reported with its interval in the resource table, where its job is to confirm that the operating point is sustained rather than nominal.

The second is what to conclude when the interval and the p-value disagree, which at modest repeat counts they routinely will: a paired interval can sit wholly on one side of zero while the corrected p-value stays far above any conventional threshold. Which instrument to believe is not a free choice, and it does not depend on whether the metric happens to be interesting. It depends on whether pairing removes the interference that particular measurement is exposed to. Pairing cancels whatever an interleaved block shared; it cannot cancel interference that landed on one run of a pair and not on the other. A run-level summary such as peak resident set size, and a central order statistic such as the median of tens of thousands of per-operation samples, are both unmoved by a handful of descheduled operations, so a paired difference that is consistent in sign across repeats is a difference in the software. A high percentile is the opposite case: p99 and p99.9 are computed from precisely the operations that were disturbed, so an interval excluding zero is as consistent with one run of a pair having been unlucky as with a real effect. Table 7.2 is therefore blocked by that distinction, and intervals in its second block are not read as findings however far from zero they sit. Applying the rule to the block that suits the argument and quietly dropping it for the other is the specific failure this pre-commitment exists to prevent.

The resolving power the design can reach is bounded by the number of repeats, and is stated rather than left implicit. An exact test on n pairs cannot return a two-sided p below $2/2^n$, and after correcting a family of m comparisons the smallest attainable adjusted value is m times that. With 15 repeats and 10 comparisons, those two floors are 0.000061 and 0.00061. Quoting them is what lets a reader tell a null result from an unresolvable one. A campaign whose corrected floor lies above the threshold being applied cannot demonstrate significance however large the effect, so reporting "not significant" from such a campaign would say more about its repeat count than about the languages. Choosing the repeat count so that the corrected floor sits below the conventional 5% is therefore part of the measurement protocol, alongside pinning the clock and isolating the cores (Section 7.1).

5.6 Reproducibility and research artifacts

The study is designed so that its results can be regenerated by a third party without access to Nokia's proprietary code. The implementations are driven entirely by the public frozen specification; the benchmarking harness, the fault catalogue and its runner, and the FFI prototype are published together with the thesis; and every figure and table in the evaluation is produced by a single script directly from the raw measurement logs, so that rerunning the campaign on a different host regenerates the artifacts verbatim. The execution environment (CPU model, kernel, compiler and toolchain versions, and the frequency and scheduling settings of the testbed) is captured automatically alongside each result set, so that the conditions of a measurement travel with its data. This separation of environment-sensitive results (the performance sweep) from environment-insensitive ones (parity testing, fault injection, and the FFI structure) is what lets the thesis label the former preliminary and the latter final without ambiguity.

5.7 Ethical and data-protection considerations

The study processes no personal, subscriber, or operator-confidential data. All traffic is synthetic and generated by the deterministic driver, the UE contexts contain only fabricated identifiers and a fixed dummy security field, and the subsystem is modelled on, rather than copied from, production code. There are therefore no privacy, consent, or data-retention concerns, and the resulting artifacts can be released openly for reproduction.

6. Design and Implementation

6.1 Shared specification: the fairness baseline

Before any code is written, a single language-agnostic specification is frozen: the public API, the UE state machine, the data structures and their algorithmic strategy (e.g. a sharded hash map keyed by UE identifier), the locking and sharding granularity, and the concurrency model. Both implementations conform to this specification so that the comparison measures language differences rather than design or skill asymmetry. The specification is itself a deliverable and is the primary control against implementation bias.

6.2 Conceptual model and concurrency design

Two functionally equivalent implementations are developed against the shared specification: a modern C++ version (C++23, STL, RAII, standard threading) and a Rust version (stable toolchain, ownership-based memory model, safe concurrency). A single concurrency model is used for both: a *shared-nothing, thread-per-shard* model that mirrors how real RAN L3 partitions UE state, and mixing of async and thread-based models is intentionally avoided.

The UE-identifier space is partitioned into N_{shards} shards (eight by default), each owned by exactly one worker thread. Only that worker reads or mutates its shard's contexts, so there is no shared mutable state between workers and no locks on context data. Callers (load-generator threads) never touch context state directly; they submit request messages to a shard's bounded multi-producer, single-consumer (MPSC) queue and wait on a single-use completion handle to which the worker writes the result. The queue is *bounded and blocking*: when a shard's queue is full, submission blocks until space frees up, which produces realistic queueing latency under overload and makes tail latency the observable of the study (a non-blocking drop policy would hide overload). Requests to the same shard are processed strictly first-in-first-out. This structure is what makes the safe-concurrency comparison meaningful: the interesting machinery is

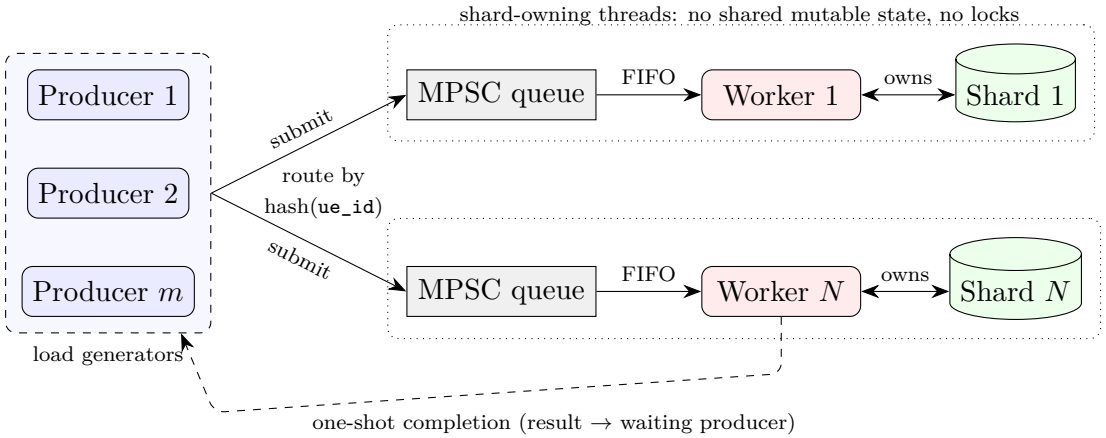


Figure 6.1: Shared-nothing, thread-per-shard architecture. Producers submit request messages to a shard's bounded, blocking MPSC queue (selected by a pinned hash of the `ue_id`) and wait on a single-use completion handle; each shard is read and written by exactly one worker thread, so there are no locks on context state. Blocking on a full queue is what makes overload observable as tail latency.

the queue, the completion handoff, and the ownership transfer of requests, rather than lock contention. The only shared mutable state permitted is a single atomic counter used to stamp a per-context activity sequence number, which is write-only bookkeeping that does not affect control flow. Figure 6.1 shows this data flow.

6.3 Architectural choices and technology stack

The C++ side uses CMake with Ninja, the standard library, and `std::thread`; the Rust side uses Cargo and the stable toolchain. To keep the comparison a language comparison rather than an allocator comparison, both link the same memory allocator: C++ links the system `jemalloc` and Rust installs a `jemalloc` global allocator (from `tikv-jemallocator`), with a system-allocator build retained on each side as a control. Both record latency through the same high-dynamic-range (HDR) histogram algorithm [58], configured identically (range $[1\,\text{ns}, 3.6\,\text{s}]$, three significant figures); Rust uses the canonical `hdrhistogram` crate and C++ uses a header-only port of the same bucketing algorithm, validated against a known distribution. Build optimisation is matched as closely as the two toolchains allow: the Rust release profile is pinned to `opt-level = 2` to match the C++ `-O2` build.

6.4 Implementation details

Both implementations follow the frozen specification field-for-field, and both are small enough to be read end to end: the C++ core is 649 lines across eight headers and two translation units, the Rust core 736 lines across eight modules; discarding blank lines and comments leaves 516 and 602 respectively. That the Rust version is the larger of the two, by 13% in raw lines and 17% in code lines, is worth stating plainly rather than leaving to a footnote, both because line count is one of the software-quality proxies this study set out to collect and because the difference is not incidental verbosity: it is the net of several forced idioms pulling in opposite directions, which Section 6.4.7 decomposes file by file. The remainder of this section works outward from the data model to the concurrency machinery, and reports the measured consequences of each language-forced divergence as it arises.

6.4.1 The UE context and its memory layout

Each UE context stores its identifier, lifecycle state (a four-value enumeration `Idle`, `Connected`, `Reconfiguring`, `Released`), serving cell, RRC transaction id, a fixed 32-byte dummy security context, a dynamic list of up to eight data radio bearers, and two counters (messages handled and an activity sequence number). The data radio bearer list is deliberately a growable container (`std::vector` / `Vec`) rather than a fixed array, so that the allocator study (RQ3) observes realistic per-context allocation. Reads return an immutable snapshot (a `UeContextView`) copied out of the worker rather than a reference into live shard state, which preserves the shared-nothing guarantee and keeps the two implementations semantically identical.

Identical field declarations do not, however, produce identical objects. Measured on the evaluation host, a `UeContext` occupies 96 bytes in C++ but only 88 bytes in Rust. The difference is entirely layout policy. C++ guarantees that non-static data members are laid out in declaration order, so the one-byte `state` enumeration sitting between the eight-byte identifier and the four-byte cell id forces three bytes of padding, and the 32-byte security key ending at offset 50 forces a further six before the eight-byte-aligned bearer vector. Rust, absent a `#[repr(C)]` annotation, is free to reorder fields and does so, packing the small scalars together and eliminating both holes. The specification deliberately does not try to erase this difference: pinning the Rust layout with `#[repr(C)]` would penalise Rust for nothing but conformity to a C++ padding rule, while hand-reordering the C++ declarations would be an optimisation the specification asks of neither side. It is instead recorded as a language-level effect and bounded, at the 5 000 active UEs used in the sweep the eight-byte gap accounts

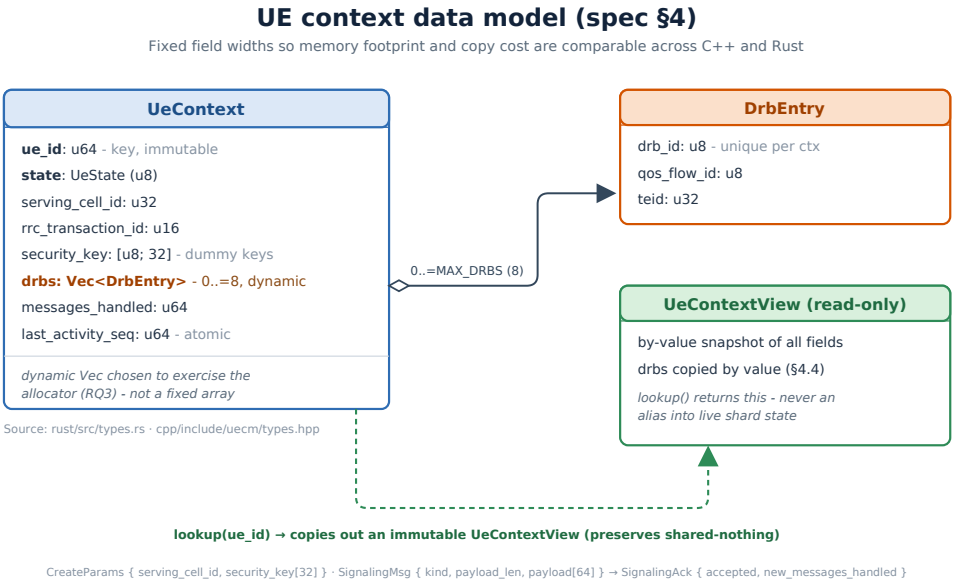


Figure 6.2: The UE-context data model shared by both implementations. Field widths are fixed so that memory footprint and copy cost are comparable across languages; the data-radio-bearer list is a dynamic `Vec`/`std::vector` on purpose, to exercise the allocator for RQ3. `lookup` returns an immutable `UeContextView` copied out by value rather than an alias into live shard state, preserving the shared-nothing guarantee.

for roughly 40 kB, against resident set sizes measured in megabytes, so it is immaterial to the RQ3 comparison.

6.4.2 The state machine as implemented

The lifecycle of Figure 3.1 is realised as a single guarded transition per operation, and Table 6.1 gives the implemented table in full. Two properties hold uniformly and are what make the two implementations comparable op for op. First, every operation validates before it mutates: the guard sequence checks the identifier, then locates the context, then tests the state, and returns the corresponding `OpError` before touching any field, so a rejected operation leaves the context bit-identical to what it was. Second, no operation can partially succeed, because each performs at most one mutation after the last guard has passed.

Two behaviours deserve explicit mention because the table alone hides them. The `Released` state is never actually stored: `release_context` erases the entry from the shard map outright, so `Released` is a transient terminal label in the specification rather than a resting value an implementation can be found in, and a second release therefore returns `NotFound` rather than `InvalidState`. And `abort_reconfigure` restores the *state* to `Connected` but does not restore the serving cell that `begin_reconfigure` overwrote with the handover target. This is a deliberate simplification, the model

Table 6.1: The implemented state-transition table, identical in both languages. Every operation additionally returns `InvalidArgument` for a zero `ue_id`, and a rejected operation leaves the context unmodified. `Released` is never stored: release erases the entry.

Operation	Required state	Resulting state	Errors
`create_context`	context absent	`Idle`	`AlreadyExists`, `CapacityExceeded`
`setup_connection`	`Idle`	`Connected`	`NotFound`, `InvalidState`
`handle_signaling`	`Connected`	`Connected`	`NotFound`, `InvalidState`
`add_drb`	`Connected`	`Connected`	`NotFound`, `InvalidState`, `CapacityExceeded`, `AlreadyExists`
`begin_reconfigure`	`Connected`	`Reconfiguring`	`NotFound`, `InvalidState`
`complete_reconfigure`	`Reconfiguring`	`Connected`	`NotFound`, `InvalidState`
`abort_reconfigure`	`Reconfiguring`	`Connected`	`NotFound`, `InvalidState`
`release_context`	any live state	entry erased	`NotFound`
`lookup`	any live state	unchanged	`NotFound`

carries no saved pre-handover cell to roll back to, but it is a semantic commitment rather than an oversight, and both implementations make it identically, which is what the parity suite of Section 6.7 checks.

6.4.3 Shard selection and the intra-shard store

To prevent the standard library hashers from becoming an accidental language advantage, both sides pin their hashing. Shard assignment applies the SplitMix64 finalizer to the `ue_id`, three xor-shift and multiply rounds with the constants `0xBF58476D1CE4E5B9` and `0x94D049BB133111EB`, before reducing modulo the shard count. The finalizer matters because the workload draws sequential identifiers, which a naive modulo would map onto shards in a fixed repeating pattern; diffusing the identifier first makes the offered load per shard uniform in both languages by construction rather than by luck. The per-shard container is each language's standard hash map (`std::unordered_map` / `std::collections::HashMap`), but the default hashers are bypassed, C++'s `std::hash<uint64_t>` is typically the identity function while Rust's default `SipHash-1-3` is cryptographically strong and correspondingly slower, so leaving them in place would have measured a standard-library choice rather than a language. Both are therefore keyed through a pinned FNV-1a 64-bit hash over the eight little-endian bytes of the identifier, and a test checks that the Rust `BuildHasher` produces a digest bit-identical to the reference `fnv1a_64`. Both maps reserve an initial capacity of 4 096 entries at start-up so that steady-state measurements are not perturbed by

rehashing. The remaining configuration is likewise shared: eight shards, a queue bound of 1 024 requests per shard, at most eight bearers and a 64-byte signaling payload per context, and a ceiling of 200 000 contexts per shard.

6.4.4 The bounded queue

The bounded blocking queue is where the two standard libraries diverge most sharply, and Listing 6.3 shows why. Rust's standard library provides the primitive the specification calls for directly: `sync_channel(capacity)` is a bounded multi-producer, single-consumer channel whose `send` blocks when the queue is full and whose `recv` blocks when it is empty, which is precisely the required backpressure semantics in a single call. The C++ standard library, through C++23, offers no bounded blocking queue at all, so the same semantics must be assembled by hand from a `mutex`, two `condition_variable`s, and a `std::queue`, and the result is the 45-line class reproduced on the left.

The point of the comparison is not that the C++ code is badly written; it is short, it uses the predicate form of `wait` so spurious wakeups are handled correctly, and it releases the lock before notifying. The point is that it exists at all. Every one of those lines is a place where a concurrency defect could be introduced, and the correctness argument for it, that the two condition variables are never confused, that the predicates match the notifications, that no wakeup is lost, is carried by the author rather than by the compiler or the library. This is the same asymmetry the fault-injection study of Chapter 7 measures, appearing here in ordinary, non-defective code rather than in a deliberately injected bug.

6.4.5 Requests, responses, and the cost of a tagged struct

The message type carried through that queue is the second forced divergence, shown in Listing 6.4. Rust models a request as an `enum` with one variant per operation; each variant carries exactly its own arguments plus a `Sender` parameterised by exactly that operation's result type. C++ has no equivalent of a payload-carrying enumeration in the standard idiom used here, so a request is a tagged struct: an `OpKind` discriminant alongside *every* argument field any operation might need, paired with a single `std::promise` whose value type must therefore be a `std::variant` over all four possible result types.

This has three measurable consequences. The first is size: a `Request` is 160 bytes in C++ against 96 in Rust, because the C++ struct always carries the create parameters, the 68-byte signaling message, the bearer entry, and the reconfiguration parameters, whether or not the operation in flight uses them, while the Rust enum is

C++: hand-written, 45 lines

```
template <class T> class BoundedQueue {
 public:
  void push(T value) {
    std::unique_lock lk(m_);
    not_full_.wait(lk, [&]{
        return q_.size() < capacity_; });
    q_.push(std::move(value));
    lk.unlock();
    not_empty_.notify_one();
  }
  T pop() {
    std::unique_lock lk(m_);
    not_empty_.wait(lk, [&]{
        return !q_.empty(); });
    T value = std::move(q_.front());
    q_.pop();
    lk.unlock();
    not_full_.notify_one();
    return value;
  }
 private:
  const std::size_t capacity_;
  std::mutex m_;
  std::condition_variable not_full_;
  std::condition_variable not_empty_;
  std::queue<T> q_;
};
```

Rust: standard library

```
let (tx, rx) =
    sync_channel(cfg.queue_capacity);

// tx.send(req) blocks when full
// rx.recv()    blocks when empty
```

Figure 6.3: The bounded blocking MPSC queue in both languages (C++ abridged only by the omission of the constructor). The semantics are identical; the C++ standard library simply does not supply the primitive, so it is assembled by hand from a mutex and two condition variables, and its correctness is argued rather than guaranteed.

C++: tagged struct + variant (160 B)

```
enum class OpKind : std::uint8_t {
  CreateContext, SetupConnection,
  HandleSignaling, AddDrb, /* ... */
  Lookup, Count, Stop };

using Response = std::variant<
    OpResult<void>,
    OpResult<SignalingAck>,
    OpResult<UeContextView>,
    OpResult<std::uint64_t>>;

struct Request {          // move-only
  OpKind kind{};
  UeId ue_id = 0;
  CreateParams create_params{};
  SignalingMsg signaling{};
  DrbEntry drb{};
  ReconfigParams reconfig{};
  std::promise<Response> completion;
};
```

Rust: enum + typed sender (96 B)

```
pub enum Request {
    Create { ue_id: UeId,
        params: CreateParams,
        resp: Sender<OpResult<()>> },
    Signal { ue_id: UeId,
        msg: SignalingMsg,
        resp: Sender<
            OpResult<SignalingAck>> },
    Lookup { ue_id: UeId,
        resp: Sender<
            OpResult<UeContextView>> },
    // ... one variant per operation
    Stop,
}
```

Figure 6.4: The request message in both languages (abridged: C++ elides four `OpKind` values, Rust seven variants). The C++ tagged struct carries every argument field on every request and returns a union that the caller must unwrap; the Rust enum carries only the arguments of the operation in flight and a sender already typed to that operation's result.

sized by its largest variant alone. With eight shards holding up to 1 024 requests each, that is roughly 1.3 MB of queue-resident request storage at full occupancy against 0.8 MB. The second is a copy cost paid on every submission, since the unused fields are default-initialised and moved along with the used ones. The third, and the most interesting for the thesis's argument, is a type-safety gap: the C++ caller must recover its typed result from the variant with `std::get<OpResult<T>>`, which throws `std::bad_variant_access` if the worker ever writes an alternative the caller did not expect. That exception cannot fire in the present code, because each `OpKind` is handled by exactly one branch that stores the matching alternative, but the invariant guaranteeing it is maintained by inspection across two files. In Rust the same invariant is discharged by construction: because the sender embedded in each variant is already typed, there is no response union, nothing to unwrap, and no runtime dispatch on the way back.

6.4.6 Service lifecycle, shutdown, and ownership

Starting the service allocates the shards, spawns one worker thread per shard, and hands each worker exclusive ownership of its store; the request path is then as described in Section 6.6. Shutdown is the more delicate half, because it must terminate workers

that are blocked on an empty queue without abandoning requests that are still in flight. Both implementations solve it the same way, with an in-band `Stop` sentinel: one `Stop` request is pushed onto each shard's queue, and because the queue is strictly FIFO, every request already enqueued ahead of it is executed and completed before the worker observes the sentinel and returns. No separate closed flag, timeout, or drain API is required, and no completion handle is left unfulfilled. The worker threads are then joined, so shutdown returns only once every shard has actually stopped.

Both languages tie this to object destruction, and the mechanism is one of the few places where the two are near-identical: the C++ destructor calls `shutdown()`, and Rust's `Drop` implementation does the same, so a service that goes out of scope cannot leave detached threads behind. Both guard against double invocation with a `running` flag, making shutdown idempotent. The remaining ownership discipline is what upholds the shared-nothing invariant end to end: a request is moved into the queue rather than copied, carrying its completion handle with it, so at no point do the producer and the worker both hold a usable reference to the same request. In C++ this is enforced by making `Request` move-only, a property it acquires automatically from the move-only `std::promise` it contains; in Rust it follows from ordinary move semantics, and the compiler rejects any attempt to use the request after submission. The only shared mutable state in either implementation is the atomic activity counter, held by reference in C++ and as a cloned `Arc<AtomicU64>` in Rust, and incremented with relaxed ordering because nothing branches on its value.

6.4.7 Forced idioms and what they cost

Where the two languages must diverge to satisfy the same requirement, the deviation is recorded in the specification's forced-idiom log so that it can be weighed rather than silently absorbed into the results. Table 6.2 collects the log in full, and the preceding subsections have already quantified the two entries with measurable consequences.

The log also explains the line-count difference reported at the head of this section, and it is more informative than the net figure suggests, because the contributions do not all point the same way. Table 6.4 decomposes the 87-line gap by concern. Rust is substantially *shorter* in exactly one place, the bounded queue, where it needs no code at all against 45 lines of C++, and longer almost everywhere else, with three contributions accounting for most of the excess.

The largest, and the one that best illustrates the trade-off, is hashing. C++ satisfies `std::unordered_map`'s hasher requirement with a five-line functor wrapping a call to `fnv1a_64`, whereas Rust must supply the whole trait apparatus: a `FnvHasher` holding the running state, a `Default` implementation seeding it with the

Table 6.2: The forced-idiom log: every point at which the two implementations diverge in order to satisfy the same specification requirement, with the measured consequence where there is one.

Concern	C++	Rust
Result type	`std::expected<T, OpError>`	`Result<T, OpError>`
Request	tagged struct: discriminant plus every argument field (160 B)	`enum`, one variant per operation, carrying only its own arguments (96 B)
Response	`std::variant` of four result types, unwrapped by the caller with `std::get`	none needed; each variant carries a sender of its own result type
Bounded queue	hand-written `mutex` plus two `condition_variable`s (45 lines)	`sync_channel(capacity)`
Completion	`std::promise` / `std::future`	a fresh `mpsc::channel()` per request
Per-shard map	`std::unordered_map` with an `FnvHash` functor	`HashMap` with an `FnvBuildHasher`
Activity counter	`std::atomic<uint64_t>` held by reference	`Arc<AtomicU64>` cloned into each worker
Teardown	destructor calls `shutdown()`	`Drop` calls `shutdown()`
Optimisation level	`-O2`	`opt-level = 2`

offset basis, a `Hasher` implementation providing `write`, `write_u64`, and `finish`, a unit `FnvBuildHasher`, and a `BuildHasher` implementation to manufacture hashers. Every line of it is boilerplate, and it buys a statically dispatched, type-checked hasher rather than a conventionally shaped callable. The service module costs a comparable amount for the reason developed in Section 6.4.5: because the C++ response is a single `variant`, all ten public operations funnel through one shared `submit` helper and each typed wrapper shrinks to two or three lines, while Rust's per-variant typed sender admits no such helper without generics, so each operation repeats the sequence of creating a completion channel, constructing its variant, sending, and receiving. The third, `lib.rs`, is the module tree and public re-export list, work that C++ gets implicitly from its header structure and `#include` graph.

None of this is a defect on either side, and none of it is visible in the performance figures. It is exactly the kind of engineering cost the forced-idiom log exists to surface: Rust spends source lines on trait machinery and call-site repetition to obtain guarantees the compiler can check, and recovers a large part of that spend wherever its standard library happens to provide a primitive that C++'s does not.

Counting lines, however, answers only half the question a reviewer of either codebase would ask, because it treats a line of boilerplate and a line of branching as the same thing. Table 6.3 therefore adds cyclomatic complexity, measured by `lizard` [62] over both cores. Applying one tool to both languages is what makes the figures comparable: it applies the same definition of a decision point on both sides, whereas each language's native tooling would count somewhat different things. The two measures point in opposite directions. Rust is 19% longer than C++ by code lines, but its total cyclomatic complexity is 15% *lower*, 79 against 93, spread over slightly more functions (40 against 37). Both cores are undemanding by the usual thresholds and both have a median per-function complexity of one, so the aggregate is carried by a small number of functions at the top.

That is where the difference sits, and it traces to a divergence already described rather than to anything diffuse. The most complex function in the C++ core is `worker_loop` at 13, more than twice its Rust counterpart's 6: it switches on the `OpKind` discriminant and then stores the matching alternative into the response `variant`, so the tagged-struct design of Section 6.4.5 concentrates the whole dispatch, and the correctness argument that goes with it, into one branch-heavy function. Rust's per-variant typed sender removes the return-path dispatch entirely, which is why its worker has nothing comparable to unwrap. The trade is therefore sharper than the line count alone suggested: the extra lines Rust spends buy back branching in the one place where branching is hardest to review, and the `std::bad_variant_access` invariant that Section 6.4.5 noted is maintained by inspection across two files lives inside precisely the

Table 6.3: Size and cyclomatic-complexity proxies for the two cores, measured with `lizard` [62] applied identically to both languages. Benchmark drivers, tests, conformance runners, and the FFI seam are excluded; only the code the parity contract constrains is counted. Complexity is one plus the number of decision points in a function. The line count is NLOC as `lizard` defines it, which also discounts brace-only and preprocessor lines and therefore runs eight below the non-blank count quoted at the head of Section 6.4 for C++; the Rust figure agrees exactly.

Measure	C++	Rust
Files	10	8
Functions	37	40
Lines of code (blank and comment lines excluded)	508	602
Total cyclomatic complexity	93	79
Mean complexity per function	2.5	2.0
Median complexity per function	1	1
Most complex function	13 (`worker_loop`)	6 (`add_drb`)

function this measurement flags.

6.4.8 The safety argument for the two cores

The Rust library contains *no* `unsafe` code at all, a property verified mechanically over the crate. Safe Rust therefore guarantees data-race freedom for this core at compile time through the `Send` and `Sync` traits: the shard store is owned by a single worker and never shared, the request type is moved across the thread boundary, and the one piece of genuinely shared state, the activity counter, is an atomic. No runtime sanitizer is needed to establish any of this, and no test coverage is required for it to hold. The C++ implementation performs no manual memory management beyond STL containers and no explicit locking outside the queue class, and it was verified data-race free under ThreadSanitizer.

The two statements are not, however, evidence of the same kind, and the difference is the thesis's central safety claim in miniature. The Rust property is a theorem about the program, discharged by the compiler on every ordinary build and covering every execution. The C++ property is an observation about the executions a particular instrumented binary happened to take, and it would have to be re-established after any change, on a build most projects do not run by default. Quantifying exactly how far apart those two kinds of assurance are is the purpose of the fault-injection study in Chapter 7.

Table 6.4: Where the 87-line difference between the two cores comes from, by concern (raw lines, including comments; C++ figures merge header and translation unit). Rust is shorter only where its standard library supplies a primitive C++ lacks.

Concern	C++	Rust	Δ
Bounded queue	45	0	−45
Hashing (functor vs. trait pair)	36	70	+34
Service (queues, workers, API)	221	255	+34
Module tree and re-exports	0	23	+23
Data model	73	85	+12
Message types	48	60	+12
Shard core (state machine)	176	188	+12
Configuration	18	29	+11
Error model	32	26	−6
Total	649	736	+87

6.5 Public API and operation semantics

Both implementations expose the same small, lifecycle-oriented API against the shared specification, so that the surface a caller sees is identical up to each language's naming conventions. A service is created with a fixed shard count and a per-shard queue bound; thereafter callers invoke context operations, create, setup, handle-signaling, add-bearer, reconfigure begin/complete/abort, release, and state lookup, each identified by the target `ue_id`. Every operation is *total* and *typed*: it returns a success value or a typed `OpError` drawn from a closed enumeration (for example `NotFound`, `InvalidState`, `BearerLimit`), and an operation that is illegal for the context's current state (Figure 3.1) returns the corresponding error and leaves the context unchanged rather than mutating it or signaling failure out of band. This choice matters for fairness as much as for correctness: because neither implementation uses exceptions or error codes as a hidden control-flow channel, the cost and the control flow of the failure path are visible and identical on both sides, and the same conformance vectors (Section 6.7) can assert op-for-op agreement on both the returned value and the resulting state.

6.6 Request lifecycle and the completion handoff

The end-to-end path of a single operation is the machinery the study actually measures, so it is worth stating precisely. A caller constructs a request carrying the operation, its arguments, and a single-use completion handle, and computes the target shard from a pinned hash of the `ue_id`. It submits the request to that shard's bounded MPSC queue:

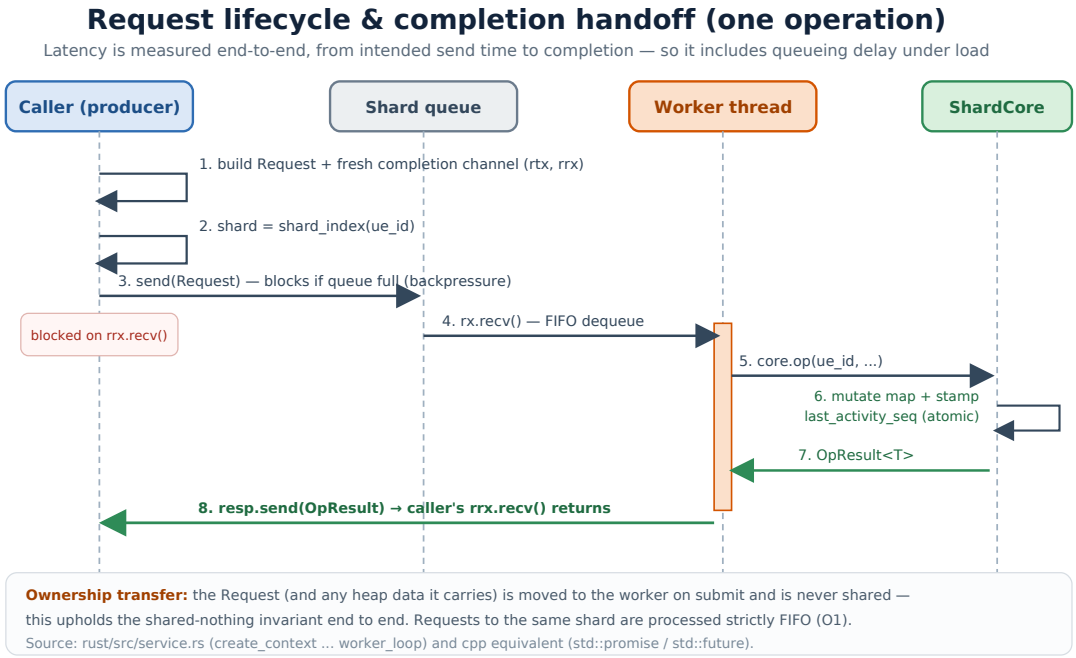


Figure 6.5: The end-to-end path of a single operation and the completion handoff. The caller builds a request with a single-use completion handle, routes it by `shard_index(ue_id)` to the owning shard's bounded queue (blocking if it is full), and waits; the shard's single worker dequeues in FIFO order, executes the operation against its exclusively owned store, and writes the result back to the completion handle. Ownership of the request is transferred to the worker on submit and never shared. Latency is measured across the whole path.

if the queue has space the request is enqueued and the caller blocks on the completion handle; if the queue is full the *submission* blocks until space frees, which is the point at which offered overload becomes observable as queueing latency rather than being silently dropped. The shard's single worker dequeues requests in strict FIFO order, executes the operation against its exclusively owned store, writes the result (a success value or an `OpError`, and for reads an immutable `UeContextView` copied out of live state) into the completion handle, and moves on to the next request. Ownership of the request, and of any heap data it carries, is transferred to the worker on submit and is never shared, which is what upholds the shared-nothing invariant end to end. Latency is measured across this entire path, from intended send time to completion, so that it includes queueing delay under load and reflects the same cross-thread round-trip on both sides.

6.7 Parity verification

Because the whole comparison rests on the two versions being functionally identical, parity is tested rather than assumed. A language-neutral conformance suite expresses test vectors as sequences of operations with their expected results and an expected post-state summary; a runner in each language executes the vectors and emits a canonical transcript, and the two transcripts are diffed for op-for-op agreement. The seeded suite covers the full error model, the reconfiguration begin/complete/abort paths, data-radio-bearer fill and overflow, and shard-assignment determinism; both implementations currently pass and agree. The benchmark workloads add a second, independent parity signal: the mixed workload's per-operation-kind counts are verified to be identical across languages for a given seed and active-UE population, confirming that both implementations are driven along exactly the same code paths.

7. Experimental Setup and Evaluation

7.1 Testbed configuration

Two execution environments must be distinguished, because the thesis deliberately reports results of two different evidential strengths and the distinction rests entirely on where they were produced.

The environment used. All results reported in this chapter were measured on a WSL2 (Windows Subsystem for Linux) development host: an eight-core x86-64 machine running the Linux kernel supplied by WSL2, with `g++ 13.3` and `rustc 1.97`. Every run captures its own environment record (CPU model, kernel, compiler and toolchain versions) alongside the measurement data, so the conditions of a measurement travel with it. This environment is entirely adequate for the environment-insensitive parts of the study, namely parity testing, the fault-injection catalogue, and the structural measurements of the FFI seam, whose outcomes are properties of the compilers and the source code rather than of the machine. It is *not* adequate for tail-latency claims: WSL2 runs under a virtualised scheduler, does not expose the CPU frequency governor or turbo state, and cannot isolate physical cores, so the far tail it produces reflects the hypervisor as much as the software under test.

The protocol the performance campaign requires. Testbed-grade latency figures therefore require a dedicated bare-metal Linux host, and the protocol for it is specified, scripted, and published so that it can be executed without further design work (`bench/TESTBED.md`). Under that protocol the benchmark process is pinned to isolated physical cores with `taskset`; the frequency governor is set to `performance`; turbo/boost and, where per-core behaviour is measured, simultaneous multithreading are disabled; and an initial warm-up fraction of each run is discarded so that cache- and allocator-cold effects do not pollute steady state. Because the analysis pipeline regenerates every figure and table from the raw logs, executing this protocol on a bare-

metal host and rerunning the same scripts is the only step separating the preliminary performance results below from final ones. That step has not been carried out, and no claim in this thesis assumes it has: the performance and resource results are labelled preliminary throughout, and the safety and migration results, which do not depend on it, are labelled final.

Measurement discipline common to both. Independently of the host, each sweep point is repeated 15 times and reported as a median with a bootstrap 95% confidence interval rather than as a single run, and the four language×allocator variants are interleaved within each repeat so that thermal or background drift affects them equally rather than biasing one. That interleaving is what makes a contended host tolerable for a *comparative* conclusion even though it invalidates the absolute numbers: interference is shared, not assigned. Throughput is counted over the measured phase alone, with the warm-up iterations excluded from both the numerator and the denominator, so that achieved throughput is directly comparable with offered load. The full configuration and scripts are published for reproducibility.

7.2 Workload and evaluation measurements

A synthetic but spec-justified telecom workload is generated by a deterministic driver shared in structure by both implementations, so that the two versions are exercised along identical code paths. Two workloads are provided. The *signal* workload preloads a population of connected UEs and then hammers the `handle_signaling` hot path, isolating steady-state per-message latency. The *mixed* workload is a pinned RAN lifecycle generator that walks each UE through a realistic procedure sequence, context creation, connection setup, signaling, bearer setup, reconfiguration/handover (begin/complete/abort), and release, using a fixed SplitMix64 draw order and a per-slot expected-state model so that every emitted operation is valid. Because the draw order is fixed, the per-operation-kind counts are identical in C++ and Rust for a given active-UE population and seed, which doubles as the parity signal described in Section 6.7. The control parameters are the target event rate and the number of active UEs, swept from nominal load through saturation and overload; a fixed send schedule with histogram correction is used so that coordinated-omission bias is avoided when the system stalls [58]. Three measurement campaigns are run: a load sweep for performance, the fault-injection study, and an FFI migration prototype.

7.3 Data collection and metrics

The study records latency predictability (per-event mean, median, p99/p99.9/p99.99, and maximum latency, plus jitter), throughput (sustained events/sec and throughput at a fixed latency service-level objective (SLO)), resource use (CPU utilisation, memory over time, allocation patterns), concurrency robustness, safety (defect classes prevented at compile time vs runtime), and software-quality proxies. The last of these are reported where they are most legible rather than gathered into this chapter: lines of code and cyclomatic complexity alongside the forced-idiom log they explain (Table 6.3), and the count and scope of Rust `unsafe` blocks alongside the FFI seam that is the only place either implementation contains any (Table 7.6). Following established guidance on statistically rigorous performance evaluation, each configuration is measured over 15 independent, interleaved repeats and reported as a median with a bootstrap 95% confidence interval rather than as a single number, so that run-to-run non-determinism does not lead to unsupported conclusions [25]; differences between languages are additionally tested on the paired repeats (Section 5.5). Only synthetic data is used, so no personal, subscriber, or confidential data is processed.

7.4 Evaluation data

The collected dataset consists of per-event latency histograms, throughput and resource-use records for each load level and language, the per-defect outcomes of the fault-injection runs, and the interop measurements of the FFI prototype. The performance campaign is a full factorial sweep of two languages $\times$ two allocators $\times$ 8 offered-load levels $\times$ 15 repeats, or 480 measured runs, captured on the development environment of Section 7.1. It is complete and self-consistent as an analysis, and is used below to answer RQ1 and RQ3 provisionally; the same scripts rerun on a bare-metal host will replace these numbers with testbed-grade ones without any change to the analysis.

7.5 Evaluation results

The performance and resource results below are preliminary (development-environment) measurements; the safety and migration results do not depend on testbed hygiene and are final.

7.5.1 Performance (RQ1, RQ3)

The hypothesis is that Rust and C++ achieve similar mean throughput but may differ measurably in tail latency and jitter under overload, and that any safety-model overhead in Rust is small in realistic workloads. The evaluation reports where, if anywhere, each language wins and by how much, across the load sweep up to and beyond saturation.

The figures and tables in this section are generated by a single reproducible script (`bench/plot_figures.py`) directly from the raw measurement logs, as are the summary numbers quoted in the text, so prose, figures, and data cannot drift apart and the whole section can be regenerated verbatim once the campaign is rerun on the bare-metal testbed of Section 7.1. The numbers reported here are *preliminary*, collected on the development environment (WSL2), which is explicitly not testbed-grade for tail-latency claims. Each point aggregates 15 interleaved repeats; tabulated values and shaded bands are medians with nonparametric bootstrap 95% confidence intervals.

Table 7.1 summarises the per-operation latency distribution at a sustained offered load of 16 kops/s. That operating point is not chosen by eye: it is the highest swept load at which every one of the four build variants still holds its p99 below 5 ms, that is, the busiest point at which all four are simultaneously healthy, which makes it the fairest place to compare them.

The table must be read at two different levels of confidence, and separating them is the honest reading of this campaign. At the *median* the four variants sit within 17 μs of one another, or 8.4%, and each variant's confidence interval contains the others' point estimates. No ordering among them survives testing, and the reason is stronger than a wide interval: the paired effect *reverses sign* between the two allocators, which is not something a language effect can do, since the allocator is not what is being varied. *Above* the median the picture is not that the languages differ but that this host cannot say whether they do. The p99 values differ by 55 μs across variants, while the widest 95% confidence interval on a single variant's own p99 spans 10975 μs, many times the between-variant difference. When the uncertainty about one variant exceeds the gap between all of them, no ordering can be read off, and reporting one would be reporting the scheduler. The same holds with greater force at p99.9 and at the maximum.

This is the concrete form of the limitation stated in Section 7.1, and it is worth being precise about what it does and does not cost. It does not affect the throughput and saturation behaviour, the resource results below, or anything in the RQ2 and RQ4 chapters. It does mean that RQ1's central question, how the two languages behave in the tail and at the overload cliff, is answered by this campaign only in the negative: the tail is not yet measurable. Closing that gap needs the bare-metal protocol rather

Table 7.1: Per-operation latency at a sustained offered load of 16 kops/s (mixed RAN-lifecycle workload, 8 shards). Values in microseconds: the median over 15 repeats above its nonparametric bootstrap 95% confidence interval. The interval is printed in full rather than as a symmetric half-width because it is strongly asymmetric at these percentiles, and a half-width wide enough to cover the upper bound would place the lower one below zero. Preliminary, development-environment (WSL2) measurements.

Variant	p50	p99	p99.9	max
C++ (system alloc)	217.2	355.1	674.3	2170
	[163.8, 237.2]	[289.3, 389.4]	[395.3, 1538.0]	[910, 5594]
C++ (jemalloc)	225.8	410.1	1188	3590
	[181.1, 261.2]	[305.7, 11280.4]	[544, 14213]	[1472, 16291]
Rust (system alloc)	214.3	376.3	1068	2404
	[191.6, 238.8]	[330.2, 3629.1]	[504, 6955]	[1129, 9667]
Rust (jemalloc)	208.4	388.9	763.4	2804
	[193.7, 223.1]	[342.3, 478.0]	[479.5, 4272.1]	[1114, 7606]

than more repeats on this host, because the dispersion is systematic interference rather than sampling noise, and Section 7.5.2 shows that the quantity a language could affect is four orders of magnitude below the one this measurement resolves.

Testing the comparison rather than eyeballing it. Reading a conclusion off overlapping confidence intervals is a weak instrument in both directions: intervals can overlap while a small difference is present in every single repeat, and they can separate without the difference being reliable. The comparison is therefore also tested directly. Because the campaign interleaves the four variants inside every repeat (Section 7.1), the runs form a matched design rather than two independent samples: the i-th run of each variant at a given load was measured in the same block, under the same ambient conditions, so whatever interference that block carried is common to the pair and cancels in their difference. This is the second thing the interleaving discipline buys, beyond the fairness argument it was adopted for.

Table 7.2 reports an exact two-sided paired permutation test of C++ against Rust for each metric, with the allocator held fixed so that language remains the only variable. With 15 repeats the 2^{15} sign assignments admitted by the null hypothesis are enumerated in full rather than sampled, so the p-values are exact and no distributional assumption is imposed on a sample already known to be skewed. The family of 10 tests is corrected with the Holm step-down procedure.

One property of the design has to be stated before the table is read, because it governs what the test can and cannot show. The smallest two-sided p-value an exact test on 15 pairs can return is 0.000061, since the observed sign assignment and its

complete negation are always at least as extreme as themselves, and after correcting for 10 comparisons the smallest attainable adjusted value is 0.00061. Both floors sit well below the conventional 5%, so a null result here is a statement about the data rather than about the repeat count. That is what the repeat count was chosen for: an earlier pass of this same campaign at seven repeats had a corrected floor above 0.05 and so could not have returned a significant result whatever the languages did, which would have made every entry in the final column uninformative.

Four things follow. First, 3 of the 10 tests survive the Holm correction: both peak resident set size rows, at the corrected floor of 0.00061, and CPU utilisation under the system allocator.

Second, the reading rule fixed in Section 5.5 has to be applied before those are interpreted, and applying it costs nothing here. 4 paired intervals exclude zero; in the tail block the intervals that do so are: none. The rule is stated in advance rather than at this point precisely because that is not guaranteed, and an earlier seven-repeat pass of the same campaign did produce tail intervals excluding zero which more repeats then removed, which is the behaviour the rule anticipates.

Third, the allocator control supplies a second and independent check, and it is the one that does real work. It was introduced so that an allocator difference could not be mistaken for a language difference, but it also means every comparison is run twice under conditions that should not change a language effect. An effect that is genuinely the language ought to keep its sign whichever allocator is linked. Only Peak RSS does: p50 latency, p99 latency, p99.9 latency and CPU utilisation all reverse between the two. That is why the Holm-significant CPU row is *not* read here as a language effect. It is significant under the system allocator and points the other way under jemalloc, and a p-value cannot rescue an effect that the experiment's own control fails to reproduce. Peak resident set size passes both tests, and Section 7.5.4 takes it up.

Fourth, the median. The paired difference there is at most 26.1 μs under either allocator, with the interval reaching 50 μs. Against a median of roughly 210 μs that reads like a tight bound, and the temptation is to report it as one: no language effect larger than a fifth of the operation. That reading would be wrong, and the reason is the subject of the next section.

7.5.2 What the end-to-end number is made of

Every latency the sweep reports is an end-to-end round-trip. A caller builds a request, routes it to a shard's bounded queue, blocks on a completion handle, and is woken once the shard's worker has run the operation and written the result back. Two costs of quite different character are bundled into that one number. One is the operation

Table 7.2: Exact paired permutation test of C++ against Rust at 16 kops/s, with the allocator held fixed. The effect is the median of the 15 paired differences (Rust − C++), shown above its bootstrap 95% confidence interval; the test statistic is their mean. p is exact over all 2^{15} sign assignments, and the final column applies the Holm correction across the whole family of 10 tests; the attainable floors are 0.000061 uncorrected and 0.00061 after correction. The rows are blocked by the criterion set out in Section 5.5: statistics in the upper block are unmoved by interference falling on a few operations within a run, so pairing rescues them and a consistent interval is evidence, whereas the high percentiles in the lower block are computed from exactly those operations, so an interval excluding zero there is not read as a finding. Achieved throughput is not tested, for the reason given in the same section, and appears in Table 7.4 instead. Preliminary, development-environment (WSL2) measurements.

Metric	Allocator	Effect (Rust − C++)	p	Holm
Robust to interference on single operations				
p50 latency (µs)	system	+26.1 [−13.3, +50.4]	0.870	1.000
p50 latency (µs)	jemalloc	−17.4 [−29.6, −12.5]	0.076	0.535
Peak RSS (MiB)	system	+2.1 [+1.9, +2.2]	0.000061	0.00061
Peak RSS (MiB)	jemalloc	+2.5 [+2.4, +2.5]	0.000061	0.00061
CPU utilisation (%)	system	+27.4 [+2.0, +42.4]	0.002	0.018
CPU utilisation (%)	jemalloc	−4.8 [−11.2, +11.0]	0.505	1.000
Determined by it: high percentiles				
p99 latency (µs)	system	+64 [−12, +663]	0.831	1.000
p99 latency (µs)	jemalloc	−22 [−10897, +145]	0.135	0.812
p99.9 latency (µs)	system	+86 [−185, +2899]	0.833	1.000
p99.9 latency (µs)	jemalloc	−32 [−13685, +1101]	0.163	0.813

Table 7.3: Where an end-to-end operation's cost actually goes, measured at 8 shards and 5 000 active UEs with a single uncontended caller. Each row is the median over 15 interleaved repeats above its bootstrap 95% confidence interval, in the unit its own magnitude calls for: the two terms differ by more than three orders of magnitude, which is the point of the table. p is the exact two-sided paired permutation test of C++ against Rust on the repeat-wise differences. The handoff is obtained as the difference between the two measured rows, so it carries the uncertainty of both. Preliminary, development-environment (WSL2) measurements.

Component	C++	Rust	p
Operation alone, no queue in the path (ns)	19.0	10.0	0.00024
	[18.7, 19.5]	[9.6, 11.2]	
Cross-thread submit-and-complete handoff (µs)	43.30	43.13	0.729
	[42.58, 45.94]	[40.60, 44.57]	
End-to-end round-trip, uncontended (µs)	43.32	43.14	0.723
	[42.60, 45.96]	[40.61, 44.58]	
Operation's share of the round-trip	0.040 %	0.020 %	

itself: a pinned hash lookup, a state-machine guard, and a field update, all executed inside a single thread and all of it code the compiler generated. The other is the cross-thread submit-and-complete handoff: an enqueue, a futex wake, a context switch, and a completion, none of it language-specific and all of it charged to the operating system. Only the first can plausibly differ between C++ and Rust, and the sweep cannot separate them.

That matters for the bound just quoted, because a bound is only as informative as the quantity it is a fraction of. The two components are therefore measured directly, in both languages, at the sweep's own shard count and context population. `ShardCore` is the single-threaded operation processor, so calling it gives the operation with no queue in the path; `Service` wraps the same core in the full threaded machinery, so calling it gives the operation plus the handoff. Their difference is the handoff. Both are batch means over a long loop rather than per-call medians, deliberately: a clock read costs tens of nanoseconds, the same order as the operation itself, so per-call timestamps would be measuring the instrument. The two languages are interleaved across 15 repeats exactly as the sweep interleaves its variants, and compared with the same exact paired test.

Table 7.3 gives the result, and it reframes the whole performance comparison.

The operation is 19.0 ns in C++ and 10.0 ns in Rust. The handoff is 43.30 μs and 43.13 μs. The work the languages actually perform is therefore at most 0.040% of what the sweep measures, and better than 99.9% of every latency in Table 7.1 is the operating system moving a message between two threads.

Two conclusions follow, and they pull in opposite directions.

The first is that the sweep cannot bound the language effect, and the bound quoted above should not be read as though it does. An interval of 50 μs on the paired median difference is 2654 times the operation's own cost. A language effect could be several hundred percent of the work being done and sit comfortably inside it. What the sweep establishes is the much weaker and still useful statement that neither language perturbs the scheduler-dominated round-trip, which is what a deployment of this shape would actually experience; what it cannot establish, at any repeat count, is anything about the code the compilers emit. That is a property of the measurement's construction rather than of the host, so unlike the tail result it will not be fixed by moving to bare metal. It is fixed by measuring the component separately, which is what this section does.

The second is that once the component *is* measured separately, the languages are not equivalent. Rust runs the bare operation about 1.9× faster, the difference is present in the great majority of repeats, and the paired test puts it below the corrected threshold. This is the only place in the whole performance campaign where a language difference in *time* is resolved at all, and the sweep is blind to it by four orders of magnitude.

The attribution deserves care, because the obvious reading is the wrong one. Nothing here implicates the borrow checker, bounds checking, or ownership. The single most likely source is already recorded in the forced-idiom log of Table 6.2: the per-shard container is each language's standard hash map, and those are different data structures. C++'s `std::unordered_map` is required by its iterator- and reference-stability guarantees to be node-based, so a lookup chases a pointer out of the bucket array into a separately allocated node, whereas Rust's `HashMap` is an open-addressed table that keeps entries inline and usually resolves a lookup within one cache line. The hash function is pinned to FNV-1a on both sides (Section 6.4) precisely so that hashing could not be the explanation, which leaves the container layout. That is a real and reportable difference between the two languages as they are actually used, since a developer reaching for the standard hash map gets whichever one their language ships. It is not evidence about Rust's safety model, and it should not be quoted as such. Confirming the mechanism would need the cache-miss counters Section 8.6 lists as the clearest instrumentation gap.

Three limits keep the finding the size it is. It is a single-threaded, uncontended microbenchmark, so it says what the operation costs and not what a loaded system does with it. It is measured on the same development host as the sweep, so the absolute figures carry the same caveat, although the ratio is far more robust than either term. And it is one operation on one data structure: generalising from it to "Rust is twice as fast" would repeat exactly the error this thesis criticises in the average-case literature.

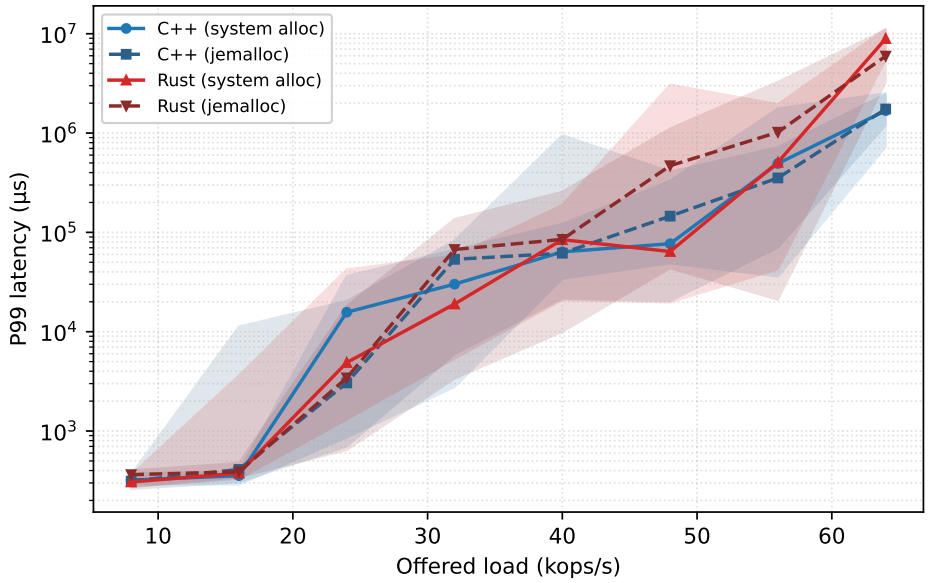


Figure 7.1: Tail latency (p99) versus offered load for the four build variants. Lines are medians over 15 repeats; shaded bands are bootstrap 95% CIs. Log-scaled y-axis. Preliminary (WSL2) data.

7.5.3 Latency, throughput, and saturation across the sweep

Figure 7.1 plots p99 latency against offered load. All four variants climb together across the sweep, by roughly three orders of magnitude between the lightest load and the heaviest, as the blocking-backpressure design pushes queueing delay into the end-to-end measurement. Two features of the plot are worth naming explicitly. The first is that the confidence bands are wide and heavily overlapping at every load level, so the curves cannot be separated from one another anywhere on the sweep. The second is what is *missing*: the sharp knee that a bounded-queue design should produce at saturation, sketched in Figure 3.3, is not cleanly visible here, because interference from other work on the host raises the tail well before the service itself is the bottleneck. The onset and shape of that overload cliff is the quantity RQ1 is really about, and this figure shows that it cannot be recovered from a shared host at any number of repeats. It is precisely what the bare-metal campaign is designed to pin down.

Figure 7.2 shows achieved throughput against offered load, and it is the more informative of the two plots on this host, because throughput is an average over a whole run and therefore far more robust to the interference that destroys the tail. Every variant tracks the ideal offered-equals-achieved line up to 40 kops/s, saturates at around 41 kops/s, and then falls back as producers block on full queues; the four are indistinguishable throughout. Two things follow. First, the saturation point is a property of the shared design rather than of either language, which is what the identical thread-per-shard structure predicts. Second, because achieved throughput

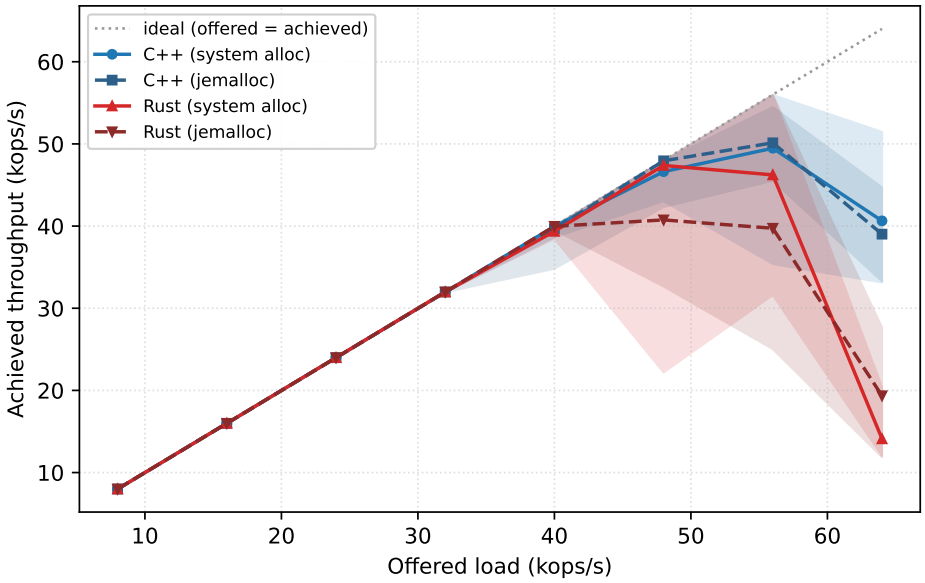


Figure 7.2: Achieved throughput versus offered load. The dotted line marks the ideal offered-equals-achieved regime. Throughput is counted over the measured phase only, so the warm-up iterations bias neither the numerator nor the denominator. Preliminary (WSL2) data.

matches offered load below the knee to within 0.1%, the operating point of Table 7.1 is genuinely a *sustained* load rather than a nominal one: the system is completing work as fast as it is offered, so the latencies reported there are not those of an already-queueing system.

Figure 7.3 isolates the shape of the distribution at the 16 kops/s sustained load by plotting latency against percentile. The curves start together at the median and fan out as the percentile rises, which is the expected shape for a heavy-tailed distribution and is exactly why a percentile-aware methodology (HDR histograms with coordinated-omission correction) is used throughout. The fanning should not be read as a language effect: as established above, the run-to-run dispersion at these percentiles is larger than the separation between the curves, so the figure shows where the measurement loses resolution rather than where the languages differ.

7.5.4 Resource use (RQ3)

Table 7.4 and Figure 7.4 report resource use at the same sustained load. Peak resident set size is modest in absolute terms across all four variants, in the single-digit to low-double-digit megabytes, but it is the one metric in the whole campaign that is cleanly resolved, and it is the only one that passes both of the checks set out above: its intervals exclude zero, and its effect keeps its sign under both allocators.

Two effects are present in it rather than one, and they are close to additive.

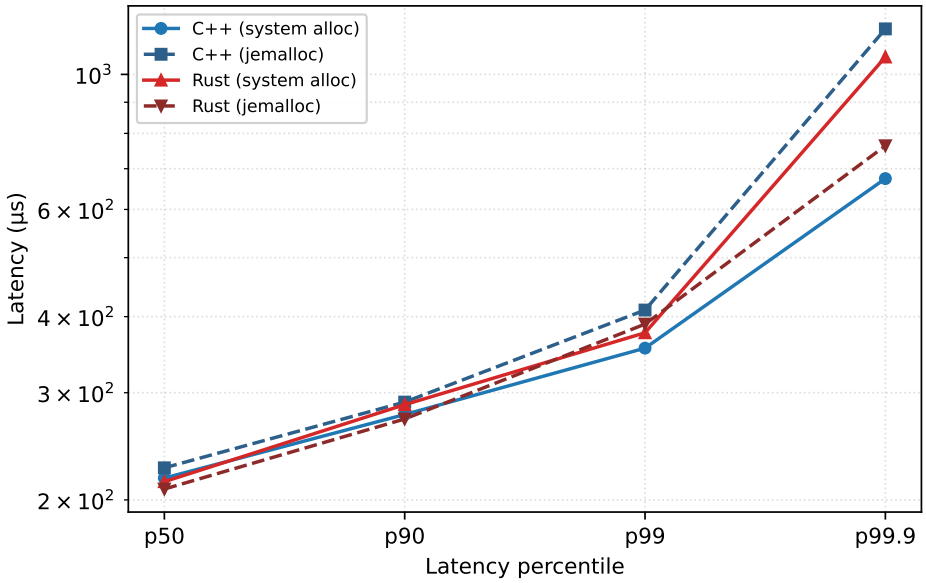


Figure 7.3: Latency by percentile at 16 kops/s sustained load; log-scaled y-axis. The four variants coincide at the median and separate as the percentile rises; on this host the separation is within run-to-run dispersion and is therefore not attributable to language or allocator. Preliminary (WSL2) data.

Linking `jemalloc` costs 2.8–3.1 MiB in either language, and the Rust build costs 2.1–2.5 MiB more than the C++ build under either allocator. Both survive the Holm correction at the corrected floor of 0.00061, and the bootstrap interval on the language difference, 1.9 to 2.5 MiB, is nowhere near zero. So resource use does not follow the allocator *rather than* the language, which is how a reading of the medians alone might put it; it follows both, with the allocator the slightly larger of the two.

Achieved throughput and CPU utilisation add nothing to this. Throughput is pinned to the offered load at a below-saturation operating point and is reported here only to confirm that the point is sustained rather than nominal, which it is to within 0.1%. CPU utilisation is the one metric in the campaign where the two instruments and the allocator control disagree with each other: the paired difference reaches 27.4 percentage points under the system allocator and clears the Holm correction there, yet it reverses sign under jemalloc, and its per-repeat spread runs to tens of percentage points in both directions. An effect that inverts when a control variable changes is not a language effect, and it is reported here as unresolved rather than as a finding. What can be said is bounded and unglamorous: no CPU difference reproduces across the allocator control.

Two qualifications keep the memory finding the size it actually is. It is a difference in fixed cost rather than in cost per context: at the 5 000 active UEs used in the sweep, the eight-byte layout difference measured in Section 6.4.1 accounts for roughly 40 kB,

Table 7.4: Resource use at a sustained offered load of 16 kops/s: achieved throughput, peak RSS, and total CPU utilisation. The median over 15 repeats above its bootstrap 95% confidence interval. Peak RSS is the one metric here whose intervals separate the four variants cleanly. Preliminary, development-environment (WSL2) measurements.

Variant	throughput (kops/s)	peak RSS (MiB)	CPU (%)
C++ (system alloc)	15.99	9.4	162.3
	[15.99, 15.99]	[9.4, 9.5]	[120.8, 184.6]
C++ (jemalloc)	15.99	12.2	172.9
	[15.99, 15.99]	[12.1, 12.4]	[139.6, 203.1]
Rust (system alloc)	15.99	11.6	178.0
	[15.99, 15.99]	[11.4, 11.6]	[150.9, 200.7]
Rust (jemalloc)	15.99	14.6	163.4
	[15.99, 15.99]	[14.5, 14.8]	[159.9, 183.5]

two orders of magnitude short of the gap, so the bulk of it is runtime and allocator-arena overhead that does not scale with the workload and would matter proportionally less, not more, in a larger deployment. And the measurement can establish the difference without explaining it: no hardware counters or allocation profiles were collected, so the split between arena pre-reservation, per-thread caches, and static footprint is not resolved here. Both points are revisited in Chapter 8.

The RQ3 finding is therefore neither the flat "no resource penalty" that overlapping intervals would suggest nor a general performance penalty. Rust costs a few megabytes more of fixed footprint, reproducibly and under both allocators; it shows no reproducible CPU difference; and on the one time-like quantity the campaign can actually resolve, the bare operation of Section 7.5.2, it is the faster of the two.

7.5.5 Safety via fault injection (RQ2)

Instead of claiming that Rust has no bugs, a catalogue of seven classic C++ defects is deliberately injected. Each is a small, self-contained program flavoured with the subsystem's domain, so that it maps onto a plausible bug in the UE Context Management service. The catalogue is deliberately built in two parts, and the second part is as important to the answer as the first.

Two groups, chosen so the catalogue can refute as well as confirm. The first five defects, use-after-free (a stale pointer into the context map), double-free (releasing a context twice), iterator invalidation (mutating a bearer list while iterating it), data race, and unguarded shared mutation (two workers updating a context field without synchronisation), lie inside the region Rust claims to govern. A catalogue containing

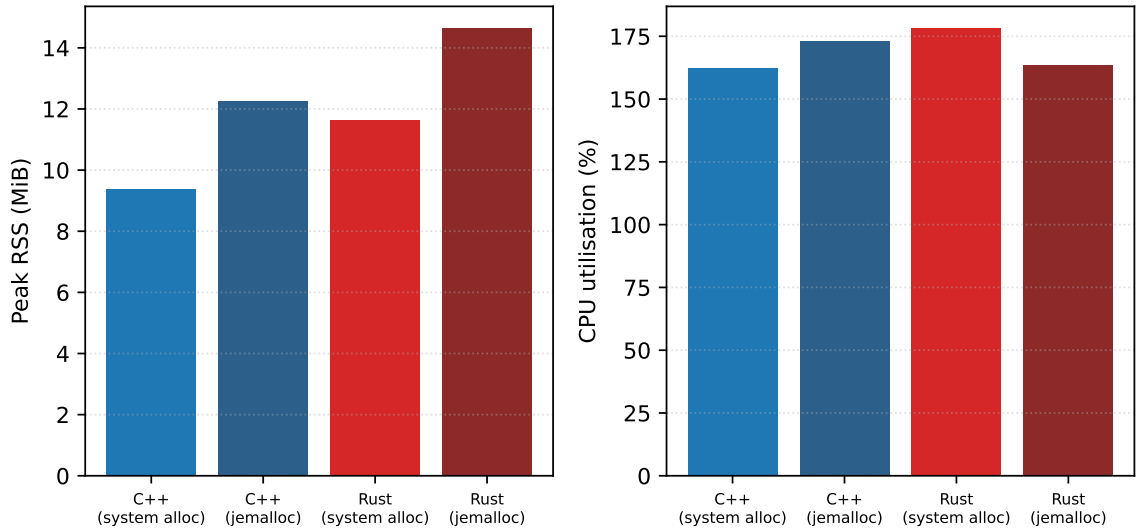


Figure 7.4: Peak resident set size (left) and CPU utilisation (right) per build variant at 16 kops/s. Preliminary (WSL2) data.

only those would be a test whose outcome is decided by the choice of test cases: it could confirm the expected result but never disturb it. Two further defects are therefore included specifically because there is good reason to expect safe Rust *not* to prevent them. A *lock-order deadlock* has two handovers acquiring the source and target context locks in opposite orders. A *lost update* has two workers increment a message counter through a read-modify-write split across two separate critical sections, so that every individual access is correctly synchronised while their composition is not atomic. Both are ordinary control-plane bugs, and neither is a memory-safety error. Including them turns the study from a demonstration into a boundary measurement: it establishes not only what the guarantee covers but where it ends. The fault runner records the expected outcome for each case and reports a violation in either direction, so a case that unexpectedly compiled, or unexpectedly failed to, would be visible rather than silently absorbed into the conclusion.

Procedure. For each C++ defect the study builds and runs the program under both AddressSanitizer (ASan) and ThreadSanitizer (TSan) and records which tool, if any, detects it. The analogous construct is written in safe Rust and compiled; where it compiles, it is then run under a bounded timeout and its observable behaviour recorded, since "it compiles" alone would not establish whether the defect survives into the running program. The catalogue, the runner, and the raw tool logs are published with the thesis.

Results. The outcomes are shown in Table 7.5, measured with `g++ 13.3` (`-std=c++23`, `-O0` so the optimizer cannot elide an injected defect) and `rustc 1.97` (edition 2021). Within its scope the result is unambiguous: every one of the five memory-safety and data-race defects is a compile error in safe Rust, so none can be written at all, whereas in C++ each is valid code whose defect manifests only at runtime, and only if a test actually exercises the buggy path. The memory-safety defects are caught by ASan; the concurrency defects are caught only by TSan and are invisible to ASan, underlining that no single C++ sanitizer covers the whole catalogue. Runtime detection further requires building a dedicated instrumented binary and paying the sanitizer's time and memory overhead, whereas the Rust errors are reported on every ordinary build regardless of test coverage. The parenthesised codes are the `rustc` diagnostics that fire (for example `E0499`, conflicting mutable borrows; `E0502`, a mutable borrow while a shared borrow is live; `E0382`, use of a moved value; `E0373`, a closure outliving borrowed data as it escapes to another thread).

The two boundary cases behave as predicted and delimit the claim. Both compile in safe Rust and both misbehave. The deadlock case hangs in both languages and had to be killed by the harness timeout; ThreadSanitizer does flag it in C++, reporting a lock-order inversion, while AddressSanitizer issues no report at all because nothing is corrupted. The lost-update case terminates in both languages with a silently wrong answer: against an expected total of 40 000 increments the safe-Rust program recorded 35 030, losing 4 970, and the C++ program under ThreadSanitizer recorded 25 448, losing 14 552.

The distinction these expose is precise, and it is the distinction the phrase "thread safety" routinely blurs. Rust's concurrency guarantee is the absence of *data races*, meaning unsynchronised concurrent access to the same location; it is not a guarantee about *lock-acquisition order* and not a guarantee about the *atomicity of a composed operation*. The lost update is the sharper of the two cases, because neither sanitiser reports it either: with every access properly locked there is no race for a race detector to find. It is therefore the one class in the catalogue that nothing in either language's toolchain addresses, and the one that fails most quietly, corrupting a counter rather than crashing. The consequence for practice is developed in Chapter 8.

Figure 7.5 presents the same outcomes as a colour-coded matrix, generated from the runner's machine-readable log (`faults/results/taxonomy.csv`), with a rule marking where the compile-time guarantee stops. Read as a whole it makes two things visible at once: above the rule the entire right-hand column is green while the C++ columns are a patchwork of runtime detections and outright misses, and below it the right-hand column turns red alongside them.

Table 7.5: Defect-class taxonomy: measured outcomes for the injected catalogue. C++ under `g++ 13.3` with ASan/TSan; Rust under `rustc 1.97`, edition 2021. "Compile error" means the safe-Rust program does not build, so the defect cannot be written. The lower block is the boundary group: defect classes chosen because safe Rust is *not* expected to prevent them, and does not.

Defect class	ASan	TSan	Safe-Rust outcome
Within the guarantee			
Use-after-free	detected	detected	compile error (E0499)
Double-free	detected	not detected	compile error (E0382)
Iterator invalidation	detected	detected	compile error (E0502)
Data race	not detected	detected	compile error (E0373†)
Unguarded shared mutation	not detected	detected	compile error (E0502†)
Boundary of the guarantee			
Lock-order deadlock	not detected‡	detected	compiles; deadlocks
Lost update	not detected	not detected	compiles; wrong result

† Several diagnostics fire for these cases; the first is shown (the data race also raises `E0382` and `E0502`, the unguarded mutation `E0524` and `E0597`). ‡ AddressSanitizer issues no report because nothing is corrupted: the instrumented binary simply deadlocks and is killed by the harness timeout.

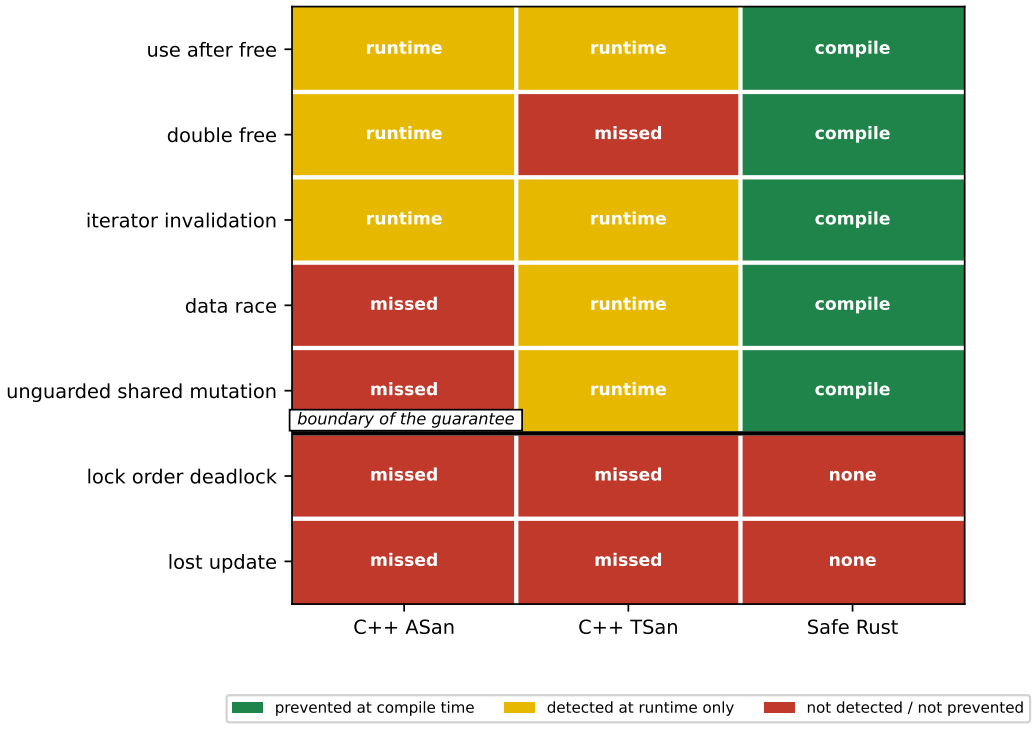


Figure 7.5: Defect-class outcome matrix, with the boundary group below the rule. Green cells are defects prevented at compile time (safe Rust); amber cells are defects that compile but are caught at runtime by the given sanitiser; red cells are defects the sanitiser does not detect or, in the right-hand column, that safe Rust does not prevent. Generated from the fault-injection runner logs.

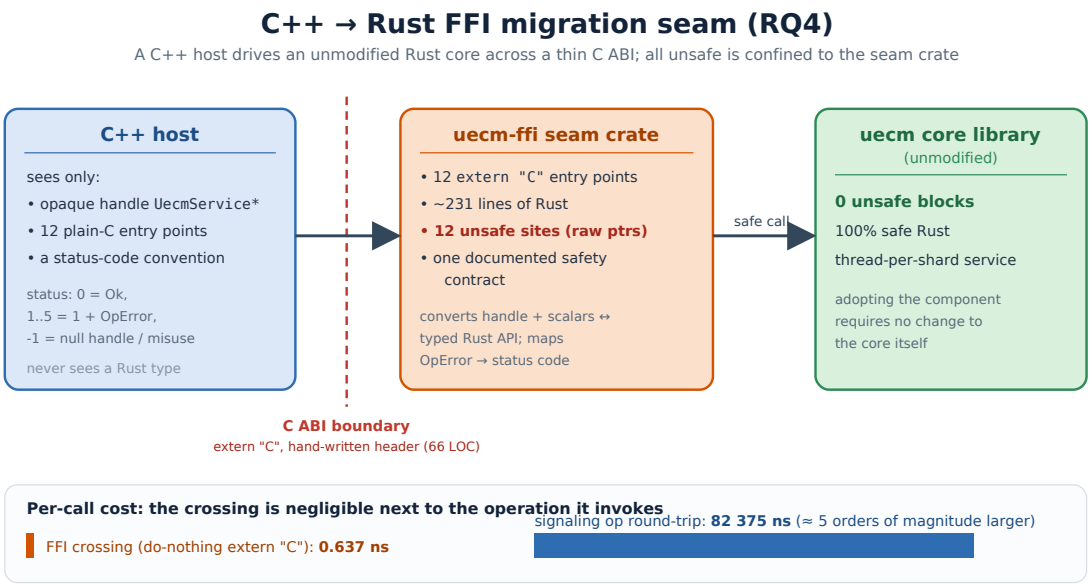


Figure 7.6: The C++-to-Rust FFI migration seam. The C++ host sees only an opaque handle, twelve plain-C entry points, and a status-code convention; a thin `extern "C"` seam crate concentrates all of the raw-pointer `unsafe` (twelve sites) and calls the unmodified Rust core, which remains free of `unsafe`. The do-nothing crossing costs a fraction of a nanosecond; measured against the same signaling operation called natively, the seam's cost per operation is bounded at 1.37%.

7.5.6 Migration (RQ4)

Incremental adoption is assessed with a working prototype that models the realistic direction of migration: an existing C++ host keeps a Rust component behind a C ABI. The Rust service is exposed through a thin `extern "C"` seam in a *separate* crate that depends on the unmodified core library, so adopting the component requires no changes to the component itself. The C++ host sees only an opaque handle and a set of plain-C entry points (a hand-written header, no generated bindings), and drives the full UE lifecycle across the boundary; the error model is transported faithfully (for example, a duplicate connection setup returns `InvalidState` through the seam).

The prototype yields concrete interop-cost figures, summarised in Table 7.6. The seam is small, twelve C-ABI entry points in roughly 230 lines of Rust plus a 66-line C header, with no third-party FFI dependency. The safety boundary is clean: the core library remains free of `unsafe`, and the raw-pointer handling the C ABI requires is confined to the seam crate (its twelve `unsafe` sites) under a single documented safety contract, so the C++ side never sees a Rust type.

The per-call cost is measured twice, because the two measurements answer different questions and only the second answers RQ4's. The first is the intrinsic crossing: a do-nothing `extern "C"` call costs 0.46 ns, an ordinary indirect call, and Figure 7.7 places it against the cost of a signaling operation. That bounds the crossing but says

Table 7.6: FFI migration prototype: seam size, safety boundary, and per-call interop cost (C++ host calling the Rust component across a C ABI, `-O2`). The core library is unmodified and `unsafe`-free; all interop is confined to the seam crate. The last three rows are the direct comparison: one signaling operation on the same service configuration and the same loop, invoked natively and then through the seam, so the difference isolates what the boundary adds rather than what the operation costs.

Metric	Value
C-ABI entry points	12
Seam size (Rust `lib.rs`)	231 LOC
Hand-written C header	66 LOC
`unsafe` in core library	0
`unsafe` in FFI seam	12
Raw FFI crossing	0.46 ns/call
Signaling op, native caller	43.2 µs/call
Signaling op, through the seam	42.4 µs/call
Seam cost per operation (paired)	-179 ns $[-2583, +594]$
as a fraction of the operation	$+1.37\%$ at the upper bound

nothing about what the seam does to a real operation, because an empty call and a full round-trip are not the same experiment: the round-trip is dominated by a cross-thread handoff, and an argument that the handoff "would be paid identically anyway" is reasoning standing in for a measurement.

The measurement is therefore made directly. The same signaling operation, on the same service configuration and in the same loop, is invoked natively from Rust and then from C++ through the seam, interleaved over 15 repeats so the pair is matched. The paired difference is -179 ns per operation with a bootstrap interval of $[-2583, +594]$ ns and an exact paired p of 0.112: no cost is detected, and more usefully, the interval bounds any cost the measurement could have missed at 1.37% of the operation. The interleaving is not optional here. A single pass of each, run while preparing this section, made the seam appear *faster* than calling natively by a third, because the run-to-run spread of the handoff on a shared host is far wider than anything a C ABI could contribute.

The main engineering effort is therefore in *designing* a C-compatible API surface, an opaque handle, scalar-typed operations, and a status convention, rather than in any runtime overhead. The reverse direction (Rust calling into C++) is symmetric but requires a C-ABI wrapper on the C++ side and re-introduces `unsafe` at the Rust caller. The prototype and its measurements are published with the thesis, and inform the migration guidelines of Chapter 8.

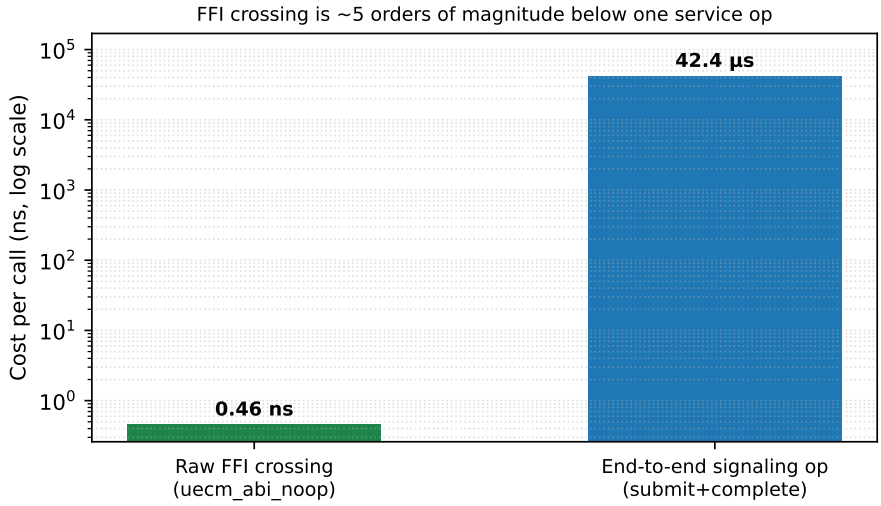


Figure 7.7: Per-call cost of crossing the C ABI (a do-nothing `extern "C"` call) versus the end-to-end cost of one signaling operation through the seam. Log-scaled y-axis: the FFI crossing is negligible next to the service's cross-thread round-trip.

8. Discussion

8.1 Summary of findings

Table 8.1 collects the headline finding for each research question together with the strength of the current evidence. The safety and migration results (RQ2, RQ4) are final, because they do not depend on the measurement environment; the performance and resource results (RQ1, RQ3) are reported from the development-environment campaign and are expected to hold on the bare-metal testbed, but are labelled preliminary until that campaign is run.

8.2 Discussion on the research questions

RQ1 (predictability). The sweep answers part of RQ1, and the decomposition of Section 7.5.2 changes what the rest of it means. End to end and below saturation the two languages are indistinguishable, agreeing to 17 μs across all four build variants, which is consistent with the wider evidence that Rust's safety checks are largely absorbed by modern out-of-order CPUs [19, 11]. That equivalence is a real and operationally relevant result: a deployment of this shape would see no difference. It is not, however, evidence that the compilers emit equally fast code, and it was tempting to report it as though it were. The operation the languages actually execute is 0.040% of the measured latency, and on that component they differ by 1.9×. An end-to-end null result in a design whose cost is dominated by a cross-thread wakeup is a statement about the design, not about the languages.

The half the sweep does not answer at all is the tail, and the reason is instructive rather than merely unfortunate. The dispersion in the p99 measurements is larger than any difference between the variants, so the data is consistent with the languages being identical in the tail and equally consistent with a substantial gap. The correct conclusion is therefore not "the languages are equivalent in the tail" but "this measurement cannot tell", and stating it that way is what keeps the eventual bare-metal result a finding rather than a confirmation of something already asserted. What the

Table 8.1: Summary of findings per research question and the current strength of evidence.

RQ	Headline finding	Evidence status
RQ1	End to end the four variants are indistinguishable at the median (17 μs, or 8.4%, apart, with the paired effect reversing sign between allocators), and no tail ordering is resolvable on this host. But the end-to-end number is 0.040% operation and the rest scheduler, so it cannot bound a language effect at all: on the operation alone, Rust is 1.9× faster.	End-to-end: preliminary (WSL2); tail not yet measurable. Operation: resolved
RQ2	All five injected memory-safety and data-race defect classes are prevented at compile time in safe Rust, where in C++ each compiles and is caught, if at all, only at runtime and no single sanitiser covers the whole catalogue. The guarantee stops there: the two boundary classes (lock-order deadlock, lost update) compile in both languages.	Final
RQ3	A memory penalty that reproduces: the Rust build carries 2.1–2.5 MiB more peak RSS under *both* allocators, Holm-adjusted $p = 0.00061$, slightly smaller than the allocator's own 2.8–3.1 MiB. No CPU effect reproduces: it clears Holm under one allocator and reverses sign under the other.	Memory: preliminary but cleanly resolved. CPU: unresolved
RQ4	The FFI seam is small and keeps the core `unsafe`-free at compile time. Measured directly against the same operation called natively, it costs −179 ns $[-2583, +594]$, bounding any tax at 1.37% of an operation; the real cost is API design. The core remains exposed at run time to memory errors in the C++ host it shares an address space with.	Final

sweep does establish is that the practical question has moved: not "is Rust fast enough on average?", which is settled here and in the literature, but "how does each language behave at the overload cliff?", which remains open.

That question is not open for want of anyone asking it. Arvidsson and Kaller measure exactly this quantity for 5G control-plane procedures and find a tail difference: their Rust runtimes match C++ with Boost.Fibers at the median but their 99th percentile degrades faster beyond a threshold request rate [8]. Taken at face value that result contradicts the expectation behind H1, and it is reported here rather than set aside. What it does not settle is attribution. The comparison is between asynchronous frameworks, and the authors trace the effect to stackless `async/await` allowing a context switch immediately before an I/O operation begins, which delays the overlap of waiting with useful work. That is a property of the runtime, not of the borrow checker, and it is the confound this study is built to exclude: both implementations here run one worker per shard over a bounded blocking queue, with no asynchronous runtime on either side. The prediction that follows is specific enough to be wrong. If tail divergence in this domain is caused by runtime scheduling, the bare-metal campaign should find the two implementations closer in the tail than Arvidsson and Kaller found their frameworks; if a comparable gap appears even with the concurrency model held fixed, the explanation has to be sought in the language, and the mechanisms already localised in the literature, bounds and overflow checking [64, 26], are where to look first.

RQ2 (safety). The result is unambiguous and does not depend on the environment, and it has two halves. Within its scope the guarantee is absolute: the borrow checker and the `Send/Sync` discipline reject all five memory-safety and data-race classes at compile time, so they cannot be written in safe Rust, whereas in C++ each is valid code detected only at runtime and only under the right sanitiser and a test that exercises the buggy path. That is a qualitative, kind-of-guarantee difference rather than a quantitative one, and it aligns with the CVE evidence that Rust memory-safety bugs invariably involve `unsafe` [61] and with the caution that unsafety can still propagate through dependencies [22], which is why keeping the core `unsafe`-free (as done here) matters.

The second half is where the guarantee stops, and it matters just as much for a vendor deciding what Rust buys. The two boundary classes compile in safe Rust and misbehave exactly as their C++ counterparts do. Both are instructive for different reasons. The lock-order deadlock shows that Rust reasons about *aliasing*, not about *acquisition order*: a `Mutex` guard is an ordinary value whose lifetime the borrow checker tracks perfectly, and tracking it perfectly says nothing about whether two threads take their locks in compatible orders. That this is a boundary of the language rather than

an artefact of the injected example is corroborated by the tooling built around it: static deadlock detection for Rust is an active research problem in its own right [63], and model checkers for Rust kernel code are motivated explicitly by the observation that ownership and borrowing leave data races, deadlocks, and livelocks reachable [57]. The lost update is sharper still, because it separates two things the phrase "thread safety" routinely conflates. Rust eliminates *data races*, unsynchronised concurrent access to a location; it does not eliminate *race conditions*, in which every individual access is correctly synchronised but the composition of them is not atomic. Here the defect is invisible to ThreadSanitizer for precisely the reason it is invisible to the borrow checker: there is no unsynchronised access to find. The practical consequence is that adopting Rust retires one category of concurrency defect completely while leaving another untouched, and the latter is the category that fails silently, corrupting a counter or a state decision rather than crashing. A migration plan that treats "memory-safe" as "concurrency-safe" would therefore mis-budget its review and testing effort, which is why this boundary is reported as a first-class result rather than a caveat.

RQ3 (overhead). The answer splits by resource, and one of the three splits is a non-answer that is worth reporting as such. On memory there is a penalty, and it is the one quantitative result in the performance campaign that the environment does not obscure. The Rust build carries 2.1–2.5 MiB more peak resident set than the C++ build, with a paired interval of 1.9 to 2.5 MiB and a Holm-adjusted p of 0.00061, slightly smaller than the allocator's own 2.8–3.1 MiB. Two independent things make it credible. Peak RSS is not a tail statistic but a run-level maximum, insensitive to the scheduling jitter that destroys the p99, so a shared host degrades it far less. And it is the only effect in the campaign that keeps its sign under both allocators, which is the replication check the design supplies for free.

On CPU there is no answer. The paired difference clears the Holm correction under the system allocator and reverses sign under jemalloc, with per-repeat spreads of tens of percentage points either way. Reporting the significant half alone would be indefensible, and reporting "no CPU penalty" would overstate a measurement that is simply too noisy on this host to say. On throughput there is nothing to measure at a below-saturation operating point, where achieved throughput tracks offered load by construction.

Three things bound what the memory result licenses. It is small in absolute terms and, more importantly, it is fixed rather than per-context, so its share of a deployment's footprint falls as the deployment grows; the measured layout difference between the two `UeContext` types contributes almost none of it. It is unexplained: attributing it between allocator arena reservation, per-thread caches, and static footprint needs

allocation profiles and hardware counters this study did not collect, which is the single clearest gap in the RQ3 instrumentation. And it should not be generalised into a claim about Rust's safety model, since nothing here isolates bounds checking or ownership-driven copying as the cause; the mechanisms that the literature has localised [64, 26] are candidates to test, not conclusions to import. What can be said is narrower and still useful to a vendor sizing a control-plane node: adopting Rust here costs a few megabytes of fixed footprint per process, and buys a faster core operation (Section 7.5.2) that this workload's structure never converts into throughput.

RQ4 (migration). The prototype shows that incremental adoption is technically cheap: a C++ host can call an unmodified Rust core across a thin C-ABI seam whose cost on a real operation is bounded at 1.37%, confining all `unsafe` to the seam. That bound comes from the direct comparison, the same operation called natively and through the seam over 15 interleaved repeats, rather than from setting an empty crossing against a full round-trip; the two are not the same experiment, and on this host a single unpaired pass of each was enough to invert the apparent ordering. The dominant cost is therefore human, not machine: designing a C-compatible surface and, per adoption studies [24, 15], climbing Rust's learning curve. This reframes migration risk as an organisational rather than a performance concern, and the two largest documented adoptions into existing C and C++ codebases report the same ordering of costs [35, 23].

One boundary on that result has to be drawn explicitly, because the natural reading overstates it. "All `unsafe` confined to the seam" is a claim about what the compiler verifies, and it is true as measured: the core crate contains no `unsafe`, and every raw-pointer operation sits in twelve reviewable sites. It is not a claim that the core is protected at run time. Host and component share an address space, and a memory error on the C++ side can corrupt Rust-owned data regardless of how the Rust side was checked, which is why the remedies proposed in the literature isolate the two heaps with hardware support rather than relying on interface discipline [53, 32]. The defect classes that arise at such boundaries have been catalogued in their own right [38]. So what a migration of this shape buys is narrower than "the Rust part is safe": it is that the migrated component stops *contributing* new memory-safety defects, while remaining exposed to those of the host until the host is migrated too. That is still the right direction of travel, and it is a reason to sequence a migration so that the components sharing the most mutable state cross over together rather than one at a time.

8.3 Revisiting the hypotheses

Chapter 4 committed to three hypotheses before any measurement was taken. Stating their disposition explicitly, rather than letting the results speak for themselves, is what distinguishes a test from a description.

H1: supported end to end; untested in the tail; too loosely worded. The hypothesis had two parts: that the languages are performance-equivalent below saturation, and that any difference is confined to the far tail. The first part is supported end to end, the four build variants agreeing to 17 μs (8.4%) at the median. The second part is not tested by this campaign, and saying so matters more than claiming it: the confidence interval on a single variant's p99 is many times the spread between all four, so the measurement is equally consistent with confinement to the tail and with a substantial tail difference.

The more interesting disposition is that the hypothesis is not well posed. "Performance-equivalent" was written as though it named one quantity, and the decomposition of Section 7.5.2 shows it names two that answer differently. At the level of the delivered operation the languages are equivalent, and that is the level a deployment experiences. At the level of the code the compilers emit they are not: Rust is 1.9× faster on the operation itself, which the end-to-end measurement dilutes by more than three orders of magnitude. Both statements are true, and a hypothesis that cannot distinguish them is one this study should have stated more carefully before measuring. H1 is recorded as supported in the sense it can be tested, untested in the sense it was really about, and ambiguous in its framing.

H2: supported in its precise form, refuted in its loose one. The hypothesis was that safe Rust prevents at compile time "the memory-safety and concurrency defect classes that C++ leaves to best-effort detection". For the five injected classes of that kind it holds without exception. The fault-injection study nonetheless refutes the broader reading the wording invites: the two boundary classes are concurrency defects, they are exactly the sort C++ leaves to best-effort detection, and safe Rust does not prevent them either. H2 is therefore accepted for memory safety and data-race freedom and rejected for concurrency correctness at large, and the latter is the reading practitioners tend to assume.

H3: supported, and tested the right way on the second attempt. The hypothesis was that interoperability cost is negligible against the operations

invoked. Measured directly, by calling the same operation natively and then through the seam over 15 interleaved repeats, the paired difference is −179 ns with an interval of [−2583, +594] ns, bounding any per-operation tax at 1.37%. The bound, rather than the failure to detect a difference, is what supports the hypothesis. The first version of this measurement did not do that: it set an empty crossing of 0.46 ns against a full round-trip and inferred the answer, which is a comparison between two different experiments and would have given the right conclusion for the wrong reason. The accompanying claim, that the real cost is engineering effort, is supported structurally rather than statistically: the seam is small and mechanical, but designing its C-compatible surface was the part of the prototype that required actual design decisions.

8.4 Engineering guidelines for incremental adoption

The results are meant to be usable, and a vendor weighing Rust for a control-plane component needs more than a set of measurements. The following guidelines are the practical form of the evidence above; each is tied to the finding that supports it, and the boundaries of that support are stated rather than glossed.

Choose the first component for its interface, not its size. The migration prototype (RQ4) bounds the run-time cost of the boundary at 1.37% of an operation, so the criterion for a first Rust component is not how small or how hot it is but how narrow and how scalar its interface is. The seam in this study stayed at twelve entry points and 231 lines because the subsystem's API was already lifecycle-oriented, typed, and free of shared mutable structures crossing the boundary. A component whose interface passes rich C++ objects, templates, or callbacks will need those designed out before it can be migrated, and that redesign, not the FFI, is the schedule risk.

Put the ABI in its own crate and keep the core `unsafe`-free. Concentrating every raw-pointer operation in a separate seam crate that depends on an unmodified core (Section 7) produced a core library with zero `unsafe` and a seam with twelve well-delimited sites under one documented safety contract. This is worth adopting as a rule rather than an accident of this prototype: it keeps the dependency-propagation problem that Evans et al. describe [22] away from the safety-critical code, and it makes the reviewable surface of the migration a fixed, countable thing. Two qualifications belong with the rule. It bounds what has to be *reviewed*, not what can be *corrupted*:

while the host remains C++, the two share a heap and the Rust side is exposed to the host's memory errors, so a team that needs the stronger property has to look at in-process isolation [53, 32] rather than at interface discipline alone. And the generated part of the seam is not above suspicion either, since the binding generators have their own defect history [14]; the seam should be reviewed as code, not trusted as tooling output.

Do not re-budget concurrency review effort on the strength of memory safety. This follows directly from the boundary cases. Adopting Rust removes use-after-free, double-free, iterator invalidation, and data races from the defect budget entirely, and those are the classes that dominate the security statistics [41, 59]. It removes nothing from the deadlock and atomicity-violation budget. Lock-ordering discipline, invariants about which locks may be held simultaneously, and review for composed-but-not-atomic operations remain exactly as necessary as they were in C++, and they should stay in the code-review checklist and the design-review agenda after migration.

Prefer a shared-nothing structure to a shared-and-locked one, in either language. The strongest concurrency result in this study is not attributable to either language: the thread-per-shard design removes locks from the hot path altogether, which is why the boundary defects had to be constructed as standalone programs rather than found in the implementations. Where state can be partitioned by a natural key, as UE state can be partitioned by `ue_id`, partitioning it converts a class of concurrency defects into a structural impossibility for both languages, and it does so without needing either compiler's help.

Budget for the learning curve, and measure it. The dominant cost identified here is human, matching what adoption studies report [24]. This thesis cannot quantify that cost, because it is a single-author study, and the guideline is correspondingly modest: treat the first component as a training investment with a deliberately generous schedule, and instrument it, recording review time and defect density, so the second component can be planned on measured rather than assumed effort.

Measure the component, not the round-trip, and do not generalise either. The performance findings are preliminary and were obtained on one subsystem, one workload, and one machine, and they carry a methodological warning that generalises further than the numbers do. An end-to-end request/response measurement of a design like this one is 0.040% language and the rest scheduler (Section 7.5.2), so it will return

"no difference" between almost any two competent implementations, and reading that as evidence about the languages is a mistake this study made before it measured the components apart. A vendor running its own comparison should budget for both measurements: the end-to-end one, which says what a deployment will feel, and the component one, which says what the language is doing. Neither substitutes for the other. Beyond that, the benchmarking framework rather than its current output is the reusable artifact: the decision it should inform is "measure this on your component", not "Rust is fast enough, proceed".

8.5 Related work

Prior Rust-versus-C/C++ comparisons focus on general-purpose performance and energy [47], and cross-language benchmark surveys report that Rust matches C and C++ on performance while adding safety [13]. Formal and empirical work establishes the soundness of Rust's safe fragment and the disciplined use of `unsafe`: RustBelt provides machine-checked foundations for the safety claims [29, 30], and the Stacked Borrows model formalises the aliasing rules that `unsafe` code must respect [28]. Empirical studies of real-world code and of all known memory-safety CVEs show that `unsafe` is used sparingly and that memory-safety defects invariably trace back to it [9, 51, 61]. A complementary study cautions that unsafety nonetheless propagates through dependency call chains, so most libraries cannot be fully statically checked [22]. Ecosystem-scale evidence has since sharpened this picture: automated auditing of the crate registry finds memory-safety defects at scale [10], a study of the advisory database attributes the bulk of reported problems to memory-safety and concurrency issues [65], and the real-world defect study cited above has been extended with a larger corpus [50]. Industry and government reports motivate memory safety from a security standpoint and now call for migration to memory-safe languages [41, 59, 17, 45, 52]; the long-standing difficulty of containing memory-corruption attacks in C/C++ with runtime mitigations alone is the security backdrop to this shift [56].

On the performance side, systems-oriented studies argue that Rust delivers safety at essentially zero runtime cost and is viable in demanding low-level domains. Rust enforces safety without garbage-collection overhead and enables efficiency techniques impractical in C [11], and a Rust kernel achieves competitive resource efficiency on constrained hardware [34]. The claim also holds where timing is tightest: embedded and real-time systems such as a Rust hypervisor gain memory safety without sacrificing performance [42], and production cloud infrastructure such as AWS Firecracker is built in Rust [6]. Closest to this domain, an idiomatic Rust user-space network driver runs within a few percent of its C reference [19], with more recent Rust packet-

processing libraries reporting similar results [60]. Where that literature reports a cost, it has begun to localise it: Rust's overhead has been attributed to specific mechanisms, chiefly bounds, overflow, and division checks [64], the bounds-check component in particular has been isolated by comparing software checks against hardware-enforced capabilities [26], and recurring performance anti-patterns in real Rust code have been catalogued [16]. Two systems studies come closest to this thesis in method rather than in domain, because they also build the same component twice and attribute the difference: a comparison of IPC mechanisms implements one network function in both C++ and Rust [37], and the RedLeaf operating system is evaluated against C baselines constructed for the purpose [43]. Neither reports behaviour past a saturation point, which is the regime this thesis targets.

A distinct strand concerns the boundary itself, and it qualifies the safety argument in a way that matters directly for incremental adoption. Confining `unsafe` to a seam does not by itself protect the Rust side: foreign code sharing an address space can corrupt Rust-owned heap data whether or not the Rust component contains `unsafe`, which is why in-process isolation schemes have been proposed to restore the guarantee [53, 32]. The defect classes specific to that seam have been catalogued for Rust/C interoperation [38], the interoperation tooling itself has been shown to carry bugs [14], and lighter-weight execution models have been proposed to keep the crossing cheap in embedded settings [54]. On how far migration can go, aliasing patterns expressible in C but not in safe Rust place a principled upper bound on translation [20], surveys of embedded Rust identify interoperation as a central open problem [55], and developer-facing studies find that `unsafe` concentrates at exactly these interoperation boundaries [40]. The largest documented incremental adoptions into existing C and C++ codebases, Rust-for-Linux [35] and the Android platform [23], report the resulting effort and friction empirically rather than anecdotally.

The predictability angle draws on the tail-latency and microsecond-scale latency literature [18, 12, 36], on work treating predictability rather than mean latency as the primary design objective [27], and on the finding that thread-per-core structuring of the kind adopted here materially affects application tail latency [21]. The RAN context draws on the O-RAN disaggregation trend [48]. Within mobile networks specifically, Rust already has a peer-reviewed precedent in the datapath, a cellular packet core built on it [49], and control-plane latency is an active target in the core network, where procedures have been redesigned for low latency [39] and procedure-level benchmarking harnesses have begun to appear [46]. The adoption angle rests on qualitative studies reporting that the main barrier to Rust is its learning curve rather than run-time cost [24], now supported by large-scale data on where learners actually stall [15] and by code-level metrics for comparing development effort across languages [7]. Measure-

ment methodology follows established guidance on statistically rigorous, repeat-based performance evaluation [25], and the fault-injection study follows standard software-fault-injection practice [44].

Two master's theses, both carried out at Ericsson, are the closest prior work by domain and deserve separate treatment. Karlbäck and Orö compare Rust and Go against C++ for RAN applications on x86 and ARM, combining runtime measurements with development-efficiency metrics, and conclude that C++ remains the choice where performance is critical [31]. Arvidsson and Kaller are closer still: they evaluate asynchronous programming frameworks, C++ with Boost.Fibers, Rust with Tokio and Monoio, Go, and Erlang, on 5G control-plane procedures, and report both median and 99th-percentile latency across a load sweep [8]. Their result is the most direct challenge in the literature to the expectation this thesis set out with: Boost.Fibers shows the lowest latency at every load, and while the Rust runtimes track it at the median, their 99th percentile rises more steeply beyond a certain request rate. That is a tail divergence in the right domain, and it would be dishonest to omit it. It is also, however, a comparison in which the concurrency runtime co-varies with the language: the authors attribute the effect to stackless `async/await` permitting a context switch immediately before an I/O operation is initiated, which is a property of Tokio and Monoio rather than of Rust. That confound is precisely what the frozen specification and the shared thread-per-shard model in this thesis are designed to remove, so the two studies are complementary rather than contradictory: theirs establishes that a tail difference is observable in this domain, and this one is constructed to determine whether the language is what causes it.

Not all of this evidence points the same way, and the thesis takes the qualifying findings seriously rather than citing only the favourable ones. Even the positive networking result records a real cost: the idiomatic Rust driver issues substantially more instructions per packet than its C reference and is measurably slower in absolute terms, its safety checks being absorbed by out-of-order execution rather than being literally free [19]; the "zero-cost" framing [11] is therefore better read as workload-dependent than as universal, which is precisely why tail behaviour must be measured on the target workload rather than assumed. The safety story is likewise not absolute: empirical studies of real-world Rust find that memory- and thread-safety bugs still occur in practice, with blocking bugs and misused synchronisation among them [51, 50], and that a substantial fraction of the ecosystem cannot be fully statically checked because `unsafe` propagates through dependency chains [22]. These qualifications do not overturn the case for Rust, but they motivate two of the thesis's design choices: measuring predictability and defect prevention directly on the subsystem instead of importing general-purpose, average-case conclusions, and keeping the safety-critical core

`unsafe`-free so that the dependency-propagation caveat does not apply to it.

Table 8.2 positions this thesis against the studies closest to it along the three axes on which it claims to differ: whether the comparison controls for design by building both systems from one specification, whether it reports tail behaviour under overload rather than average-case performance, and whether it measures the cost of adoption rather than describing it. Individually, none of the three axes is unoccupied: matched implementations exist in systems software [19, 37, 43], percentile latency has been reported for 5G control-plane procedures [8], and development effort has been measured for RAN applications [31]. No prior study combines the three for a stateful, concurrent control-plane workload, which is the narrower gap this work occupies.

8.6 Threats to validity and mitigations

Because every conclusion here attributes a difference to the implementation language, the credibility of the study rests on the controls that hold everything else equal and on an honest account of where those controls stop. The threats are grouped below by the kind of inference each endangers.

Internal validity: is the language really the only difference? The central threat is implementation bias between the two versions. It is mitigated structurally rather than by assertion: the specification is frozen before any code is written (Section 6.1) and fixes the API, state machine, data structures, algorithmic strategy, and concurrency model; both sides link the same allocator and matched optimisation levels; both pin the same shard-selection and intra-shard hash functions so that neither standard library's hasher becomes an accidental advantage; and functional equivalence is tested rather than assumed by diffing canonical transcripts from a language-neutral conformance suite (Section 6.7). Where the two languages cannot express the same construct, the divergence is recorded in the forced-idiom log instead of being smoothed over. What remains uncontrolled is single-author skill asymmetry: one person wrote both implementations and is not equally expert in both, so a residual advantage to whichever language that person knows better cannot be excluded. The parity suite bounds this threat for *behaviour* but not for *performance*, and the honest statement is that it is mitigated, not eliminated.

Construct validity: do the metrics measure what the questions ask? RQ1 asks about predictability and is answered with high percentiles, the maximum, and jitter rather than the mean, which is the standard operationalisation in this literature [18]; the residual risk is that p99.9 on a synthetic workload is not the same quantity as a

Table 8.2: Positioning against the closest prior work. "Matched pair" means both systems are built from one specification so that the language is the only systematic difference; "tail under overload" means high percentiles are reported past the saturation point rather than average or nominal-load performance; "adoption cost" means the cost of incremental migration is measured rather than surveyed. "Partial" marks percentiles reported across a load sweep that stops at a latency requirement rather than continuing past saturation.

Study	Domain	Matched pair	Overload tail	Adoption cost
Pereira et al. [47]	General benchmarks	no	no	no
Bugden & Alahmar [13]	General benchmarks	no	no	no
Balasubramanian et al. [11]	OS / systems	no	no	no
Emmerich et al. [19]	Packet forwarding	yes	no	no
Li et al. [37]	Network function / IPC	yes	no	no
Narayanan et al. [43]	OS / device drivers	yes	no	no
Mo et al. [42]	Embedded hypervisor	no	no	no
Qazi et al. [49]	Cellular packet core	no	no	no
Qin et al. [51]	Rust ecosystem	n/a	no	no
Evans et al. [22]	Rust ecosystem	n/a	no	no
Fulton et al. [24]	Developer study	n/a	no	surveyed
Karlbäck & Orö [31]	RAN applications	no	no	dev. effort
Arvidsson & Kaller [8]	5G control plane	no	partial	no
This thesis	RAN control plane	yes	yes	measured

missed procedure deadline in a live network. A second and larger construct gap in RQ1 was found rather than anticipated: the end-to-end latency the sweep reports is 0.040% operation and the rest cross-thread handoff (Section 7.5.2), so it measures the delivered cost of the design faithfully and the cost of the language barely at all. Which of those RQ1 is asking about depends on the reader: a vendor sizing a node wants the first, a language comparison wants the second. The study now reports both and keeps them apart, but the question as originally worded did not distinguish them. RQ2 asks which defect classes a language prevents and is answered by a catalogue of seven hand-written programs; a catalogue is a sample, and a sample chosen by the author could flatter either language. Including the two boundary classes is the direct mitigation, since it forces the catalogue to contain cases the expected conclusion does not cover, but seven programs remain seven programs. RQ3 measures peak RSS and aggregate CPU, which capture resource cost but not its origin: no hardware counters, allocation profiles, or cache statistics were collected, so the study can say that no language-level penalty appeared without being able to say why; the mechanisms most likely responsible have been isolated elsewhere [64, 26], and confirming which of them applies here would need counter-level instrumentation this study did not collect. RQ4 measures the size of the seam, the `unsafe` it concentrates, and the per-call cost, and these are the right quantities for the question as posed; they are not, however, a measure of how safe the composed system is. Because host and component share an address space, a compile-time property of the Rust side does not bound what the C++ side can corrupt at run time [53, 32], so the construct measured here is the cost and reviewability of the boundary rather than the safety of the whole.

Conclusion validity: are the differences real or noise? Every configuration is measured over 15 interleaved repeats and reported as a median with a nonparametric bootstrap 95% confidence interval rather than a point estimate, in line with established guidance [25], and latency is captured in high-dynamic-range histograms against an intended send schedule so that coordinated omission does not hide the tail [58]. Comparisons are not left to interval overlap: the interleaved design is exploited as a matched one and tested with an exact paired permutation test, Holm-corrected across the family (Section 5.5, Table 7.2).

The repeat count is chosen so that the design is not the binding constraint. An exact test on 15 pairs floors at 0.000061, and at 0.00061 after correcting 10 comparisons; both sit far below the conventional threshold, so a null result here reflects the data rather than the arithmetic. An earlier pass at seven repeats did not have that property, and its corrected floor above 0.05 made every null in the table uninterpretable.

Three limitations remain. First, the measurements were taken under a virtu-

alised scheduler that does not expose frequency or core-isolation control, so the far tail contains a contribution from the hypervisor that cannot be separated from the software under test. This is why the RQ1 and RQ3 conclusions are labelled preliminary throughout rather than qualified in a footnote. It interacts with the pairing in a way worth naming: pairing removes the interference that is *common* to a repeat, which is why a run-level statistic such as peak RSS comes through cleanly, but it cannot remove interference that lands on one run and not its partner, which is exactly the regime the tail percentiles live in.

Second, a corrected p-value below 0.05 is necessary here but visibly not sufficient. CPU utilisation clears the Holm correction under one allocator and reverses sign under the other, which no threshold would have caught, and which is only visible because the design happens to run every comparison twice. Where the two disagree this thesis follows the replication rather than the p-value, and that rule is applied to CPU, where it costs a result, as well as to peak RSS, where it confirms one. A design without such a control would have reported the CPU effect.

Third, the end-to-end measurement is the wrong instrument for part of what RQ1 and RQ3 ask, and no amount of statistical care fixes that. Section 7.5.2 shows the quantity a language could affect to be 0.040% of the quantity being measured, so a null result end to end is close to guaranteed regardless of the languages. This is a construct limitation rather than a conclusion-validity one, but it belongs here because it is what a reader would otherwise take the null results to mean. The remedy used is to measure the component separately; the residual risk is that a microbenchmark of one operation is a narrower object than the sweep it supplements.

External validity: how far do the results travel? The study covers one subsystem, one workload, one concurrency structure, and one machine, and each of those bounds the reach of the conclusions differently. The safety and migration results (RQ2, RQ4) generalise most readily, because they are properties of the compilers and of the C ABI rather than of this subsystem: any C++ codebase writing these defect classes gets the same compile-time outcome, and any narrow, scalar-typed interface gets a similarly cheap seam. The performance results generalise least. UECM was deliberately chosen to be small and partitionable, and its shared-nothing design removes lock contention from the hot path; a subsystem with genuinely shared mutable state, deep call graphs, or heavy allocation churn could exercise the two languages' abstraction and allocation costs quite differently. The workload is synthetic and spec-justified rather than replayed from production traffic, so its operation mix is plausible rather than validated. And nothing here speaks to PHY or L2 processing, to the data-heavy user plane, or to the operational concerns (upgrade, observability, multi-tenancy) that a production

deployment would raise. The defensible claim is therefore narrow: for stateful, concurrent, partitionable control-plane services of this kind, the language choice is not a performance decision. Extending it beyond that class requires rerunning the framework, which is why the framework rather than the numbers is offered as the reusable contribution.

8.7 Ethical and professional reflection

The conduct of the study raised no data-protection questions, for the reasons set out in Chapter 5: all traffic is synthetic, the contexts carry fabricated identifiers, and the subsystem is modelled on rather than copied from production code. The *use* of the study is a different matter and deserves stating explicitly, because a language-selection recommendation in this domain is not a neutral technical output.

Two responsibilities follow. The first concerns over-claiming. A vendor could reasonably read the RQ2 result as licence to describe a migrated component as "safe", and the boundary cases are the reason that reading would be wrong: a deadlocked control plane is an outage, and a silently lost counter update can misinform a mobility decision, and neither is prevented by anything demonstrated here. Reporting the boundary as prominently as the guarantee is a professional obligation, not a stylistic choice, precisely because the audience for this work is making a decision that affects networks people depend on. The second concerns the weight placed on preliminary evidence. Tail-latency numbers measured under a hypervisor are not a sound basis for a production commitment, and labelling them as preliminary in the text is only effective if the label travels with the numbers when they are quoted elsewhere; that is part of why the summary table records an evidence status per research question rather than a finding alone. Beyond the immediate results, a migration recommendation also carries organisational consequences, retraining, hiring, and a period of reduced velocity, that fall on people rather than on systems, and the honest framing of the RQ4 finding is that the cost was relocated from run time to human effort rather than removed.

9. Conclusion and Future Work

This thesis compared modern C++ and Rust on a UE Context Management service modelled on a real Nokia gNB-CU-CP component, along four dimensions: tail-latency predictability, compile-time defect prevention, safety-model overhead, and incremental-adoption cost. Built against a single frozen specification and driven through identical code paths, the two implementations let language effects be isolated from design and skill differences.

Three conclusions follow from the evidence. First, on safety the difference is one of *kind*, and it has a definite edge. Safe Rust rejects all five injected memory-safety and data-race classes at compile time, turning runtime hazards that C++ can only hope to catch with sanitiser-instrumented tests into errors that cannot be written at all; but the two boundary classes, a lock-order deadlock and a lost update, compile and misbehave in safe Rust exactly as they do in C++. The guarantee is therefore best described precisely: Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors. Second, on migration the cost is dominated by API design and the learning curve rather than by any run-time tax: an unmodified Rust core can be driven from C++ across a thin, `unsafe`-isolated seam whose cost on a real operation, measured against the same operation called natively, is bounded at 1.37%.

Third, on performance the evidence divides by *what is being measured* before it divides by resource, and getting that order right was the study's most consequential correction to itself. An end-to-end operation in this design is 0.040% the work the language performs and the rest a cross-thread handoff the operating system performs. End to end, then, the two languages are equivalent at the median and their tail is unresolvable on a shared host, and the first of those is a real result about deployments of this shape while the second is an admission about the instrument. But an end-to-end null cannot bound a language effect: the interval it supports is some three orders of magnitude wider than the work it would have to constrain. Measured on the operation alone, the languages are not equivalent, and the faster of the two is Rust, by about a factor of two, for reasons that trace to the standard library's choice of hash-map layout rather than to anything about safety. On memory the ordering is the other way and it reproduces: the Rust build carries a few MiB more fixed footprint under

both allocators. On CPU nothing reproduces at all. Running the specified bare-metal protocol is therefore still the single most important remaining step for the tail, and the published scripts regenerate every figure, table, and quoted number once that data is captured.

Taken together these support a narrow but decision-grade claim. For stateful, concurrent, partitionable control-plane services of this kind, the choice between modern C++ and Rust is not a performance decision; it is a decision about which classes of defect an organisation wants to make unwriteable, and about whether it is prepared to pay the engineering and learning cost of getting them. What this thesis adds to that decision is not a verdict but an instrument: a frozen specification, a parity-tested pair of implementations, a reproducible measurement framework, and a defect catalogue that maps both what the guarantee covers and where it stops.

Several directions extend the work, each of them the direct remedy to a limitation named above. The most immediate is executing the bare-metal campaign that turns the RQ1 and RQ3 results from preliminary into final. Its target is now narrower than it was: the repeat count is no longer the binding constraint, and the end-to-end sweep is known not to be the instrument that can answer the language question, so what bare metal buys specifically is the tail and the overload cliff. Beyond that:

- Replay captured, anonymised production traces in place of the synthetic workload, so that the operation mix is validated rather than merely spec-justified.

- Scale the comparison to further control-plane subsystems, or to the data-heavy L2/PHY path, where shared mutable state and allocation churn would exercise the two languages differently than a shared-nothing design does.

- Instrument the comparison with hardware performance counters and allocation profiles. This is what would turn the memory result of Section 7.5.4 from an observation into an explanation, and what would confirm or refute the cache-locality account offered in Section 7.5.2 for the operation-level gap. It is the clearest instrumentation gap in the study as it stands.

- Extend the fault catalogue further along the boundary mapped here, towards leaks, panics, and misuse of `unsafe`.

- Evaluate Rust async runtimes against the thread-per-shard model used here. That would separate the tail divergence Arvidsson and Kaller attribute to stackless `async`/`await` [8] from the language itself, under the controls this study imposes.

- Measure what in-process isolation of the two heaps costs at the seam, since that is what an `unsafe`-free core would need in order to be protected as well as verified [53, 32].
- Track defect density and review effort in a longer-term case study, as Rust is adopted incrementally in a production RAN codebase.

Bibliography

[1] 3GPP. NG-RAN; architecture description. Technical Specification (TS) 38.401, 3rd Generation Partnership Project (3GPP), 2022. V17.5.0, Release 17.

[2] 3GPP. NR; NR and NG-RAN overall description; stage 2. Technical Specification (TS) 38.300, 3rd Generation Partnership Project (3GPP), 2022. V17.5.0, Release 17.

[3] 3GPP. NG-RAN; E1 application protocol (E1AP). Technical Specification (TS) 38.463, 3rd Generation Partnership Project (3GPP), 2023. V17.5.0, Release 17.

[4] 3GPP. NR; radio resource control (RRC); protocol specification. Technical Specification (TS) 38.331, 3rd Generation Partnership Project (3GPP), 2023. V17.5.0, Release 17.

[5] 3GPP. Service requirements for the 5G system; stage 1. Technical Specification (TS) 22.261, 3rd Generation Partnership Project (3GPP), 2023. V18.6.0, Release 18.

[6] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: Lightweight virtualization for serverless applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 419–434. USENIX Association, 2020.

[7] L. Ardito, L. Barbato, R. Coppola, and M. Valsesia. Evaluation of Rust code verbosity, understandability and complexity. *PeerJ Computer Science*, 7:e406, 2021.

[8] A. Arvidsson and M. Kaller. Evaluating asynchronous programming frameworks for 5G control plane procedures. Master's thesis, Linköping University, Department of Computer and Information Science, 2025. Carried out at Ericsson AB.

[9] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers. How do programmers use unsafe Rust? *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):136:1–136:27, 2020.

[10] Y. Bae, Y. Kim, A. Askar, J. Lim, and T. Kim. Rudra: Finding memory safety bugs in Rust at the ecosystem scale. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, pages 84–99, 2021.

[11] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. System programming in Rust: Beyond safety. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*, pages 156–161, 2017.

[12] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. Attack of the killer microseconds. *Communications of the ACM*, 60(4):48–54, 2017.

[13] W. Bugden and A. Alahmar. Rust: The programming language for safety and performance. *arXiv preprint arXiv:2206.05503*, 2022.

[14] X. Cai, J. Liu, C. Liu, L. Bao, Y. Yu, and L. Jiang. Fortifying the seams between C/C++ and Rust: Characterizing bugs in interop tools. *ACM Transactions on Software Engineering and Methodology*, 2026.

[15] W. Crichton and S. Krishnamurthi. Profiling programming language learning. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):29–54, 2024.

[16] C. Cui and H. Xu. Unleashing the efficiency of Rust: An empirical study of performance bugs in Rust projects. In *Proceedings of the 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, pages 371–381, 2025.

[17] Cybersecurity and Infrastructure Security Agency and National Security Agency and Federal Bureau of Investigation and others. The case for memory safe roadmaps: Why both C-suite executives and technical experts need to take memory safe coding seriously. Cybersecurity information sheet, CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023.

[18] J. Dean and L. A. Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[19] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. The case for writing network drivers in high-level programming languages. In *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2019.

[20] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf. Aliasing limits on translating C to safe Rust. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):551–579, 2023.

[21] P. Enberg, A. Rao, and S. Tarkoma. The impact of thread-per-core architecture on application tail latency. In *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, pages 1–8, 2019.

[22] A. N. Evans, B. Campbell, and M. L. Soffa. Is Rust used safely by software developers? In *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, pages 246–257, 2020.

[23] Y. Fan, C. Wang, Z. Dong, T. Liu, and H. Wang. An empirical analysis of Rust integration in the Android open source project. In *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, pages 545–556, 2026.

[24] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. Benefits and drawbacks of adopting a secure programming language: Rust as a case study. In *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*, pages 597–616. USENIX Association, 2021.

[25] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.

[26] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. Rust for Morello: Always-on memory safety, even in unsafe code (experience paper). In *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*, volume 263 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 39:1–39:27, 2023.

[27] J. Huang, B. Mozafari, G. Schoenebeck, and T. F. Wenisch. A top-down approach to achieving performance predictability in database systems. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pages 745–758, 2017.

[28] R. Jung, H.-H. Dang, J. Kang, and D. Dreyer. Stacked borrows: An aliasing model for Rust. *Proceedings of the ACM on Programming Languages*, 4(POPL):41:1–41:32, 2020.

[29] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018.

[30] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. Safe systems programming in Rust. *Communications of the ACM*, 64(4):144–152, 2021.

[31] R. Karlbäck and A. Orö. Holistic view on alternative programming languages for radio access network applications in cloud and embedded deployments. Master's thesis, Linköping University, Department of Computer and Information Science, 2021. Carried out at Ericsson AB.

[32] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. PKRU-Safe: Automatically locking down the heap between safe and unsafe languages. In *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*, pages 132–148, 2022.

[33] S. Klabnik and C. Nichols. *The Rust Programming Language*. No Starch Press, San Francisco, CA, 2nd edition, 2023.

[34] A. Levy, B. Campbell, B. Ghena, P. Pannuto, P. Dutta, and P. Levis. The case for writing a kernel in Rust. In *Proceedings of the 8th Asia-Pacific Workshop on Systems (APSys)*, pages 1–7, 2017.

[35] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu. An empirical study of Rust-for-Linux: The success, dissatisfaction, and compromise. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*, pages 425–443. USENIX Association, 2024.

[36] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble. Tales of the tail: Hardware, OS, and application-level sources of tail latency. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, pages 9:1–9:14, 2014.

[37] Z. Li, T. Huang, V. Narayanan, and A. Burtsev. Understanding the overheads of hardware and language-based IPC mechanisms. In *Proceedings of the 11th Workshop on Programming Languages and Operating Systems (PLOS)*, pages 53–61, 2021.

[38] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. Detecting cross-language memory management issues in Rust. In *Computer Security – ESORICS 2022*, pages 680–700, 2022.

[39] Y.-S. Liu, S. Qi, P.-Y. Lin, H.-S. Tsai, K. K. Ramakrishnan, and J.-C. Chen. L25GC+: An improved, 3GPP-compliant 5G core for low-latency control plane operations. In *Proceedings of the 2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*, pages 203–211, 2023.

[40] I. McCormack, T. Dougan, S. Estep, H. Hibshi, J. Aldrich, and J. Sunshine. A mixed methods study on the implications of unsafe Rust for interoperation, encapsulation, and tooling. *ACM Transactions on Software Engineering and Methodology*, 2026.

[41] M. Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. BlueHat IL, Microsoft Security Response Center (MSRC), 2019. Approximately 70% of Microsoft CVEs are memory-safety issues.

[42] C. Mo, L. Wang, S. Li, K. Hu, and B. Jiang. Rust-Shyper: A reliable embedded hypervisor supporting VM migration and hypervisor live-update. *Journal of Systems Architecture*, 142:102948, 2023.

[43] V. Narayanan, T. Huang, D. Detweiler, D. Appel, Z. Li, G. Zellweger, and A. Burtsev. RedLeaf: Isolation and communication in a safe operating system. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 21–39. USENIX Association, 2020.

[44] R. Natella, D. Cotroneo, and H. S. Madeira. Assessing dependability with software fault injection: A survey. *ACM Computing Surveys*, 48(3):44:1–44:55, 2016.

[45] Office of the National Cyber Director. Back to the building blocks: A path toward secure and measurable software. Technical report, The White House, Office of the National Cyber Director (ONCD), Washington, DC, Feb. 2024.

[46] C. Park and J. Nam. CtrlBench: Towards accurate API-level benchmarking in 5G core control plane. In *Proceedings of the 2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*, pages 1832–1837, 2025.

[47] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pages 256–267, 2017.

[48] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials*, 25(2):1376–1411, 2023.

[49] Z. A. Qazi, M. Walls, A. Panda, V. Sekar, S. Ratnasamy, and S. Shenker. A high performance packet core for next generation cellular networks. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*, pages 348–361, 2017.

[50] B. Qin, Y. Chen, H. Liu, H. Zhang, Q. Wen, L. Song, and Y. Zhang. Understanding and detecting real-world safety issues in Rust. *IEEE Transactions on Software Engineering*, 50(6):1306–1324, 2024.

[51] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. Understanding memory and thread safety practices and issues in real-world Rust programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 763–779, 2020.

[52] A. Rebert and C. Kern. Secure by design: Google's perspective on memory safety. Google Security Engineering, Technical Report, 2024.

[53] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. Keeping safe Rust safe with Galeed. In *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*, pages 824–836, 2021.

[54] L. Schuermann, A. Thomas, and A. Levy. Encapsulated functions: Fortifying Rust's FFI in embedded systems. In *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification (KISV)*, pages 41–48, 2023.

[55] A. Sharma, S. Sharma, S. R. Tanksalkar, S. Torres-Arias, and A. Machiry. Rust for embedded systems: Current state and open problems. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2296–2310, 2024.

[56] L. Szekeres, M. Payer, T. Wei, and D. Song. SoK: Eternal war in memory. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*, pages 48–62, 2013.

[57] R. Tang, M. Wang, X. Sun, L. Huang, Y. Huang, and X. Ma. Converos: Practical model checking for verifying Rust OS kernel concurrency. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*, pages 1143–1159. USENIX Association, 2025.

[58] G. Tene. HdrHistogram: A high dynamic range histogram, and the coordinated omission problem. http://hdrhistogram.org/, 2015. Accessed 2026.

[59] J. Vander Stoep and S. Hines. Memory safe languages in Android 13. Google Security Blog, 2022.

[60] Y. Xiao, Y. Chen, X. Zhao, J. Duan, X. Zhang, W. Li, and X. Tao. rpkt: A generic, safe, and efficient userspace packet processing library in Rust. In *Network and Parallel Computing (NPC)*, pages 454–465, 2024.

[61] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. Memory-safety challenge considered solved? an in-depth study with all Rust CVEs. *ACM Transactions on Software Engineering and Methodology*, 31(1):3:1–3:25, 2021.

[62] T. Yin. Lizard: An extensible cyclomatic complexity analyzer for many programming languages. http://www.lizard.ws/, 2025. Version 1.23.0. Accessed 2026.

[63] Y. Zhang, K. Zhang, and G. Liu. Static deadlock detection for Rust programs. arXiv preprint arXiv:2401.01114, 2024.

[64] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. Towards understanding the runtime performance of Rust. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 140:1–140:6, 2022.

[65] X. Zheng, Z. Wan, Y. Zhang, R. Chang, and D. Lo. A closer look at the security risks in the Rust ecosystem. *ACM Transactions on Software Engineering and Methodology*, 33(2):34:1–34:30, 2024.

Appendix A. Fault-injection example

As an illustration of the fault-injection study (Chapter 7, RQ2), the use-after-free defect is shown in both languages. In C++ the alias into the context map survives the `erase` that frees it, so the following program compiles cleanly and is flagged only at runtime by AddressSanitizer:

```
std::unordered_map<unsigned long, UeContext> ctxs;
ctxs[1001] = UeContext{1001, 0};
UeContext* p = &ctxs.at(1001);  // alias into the container
ctxs.erase(1001);               // frees the node that p refers to
p->messages_handled += 1;       // use-after-free
```

The analogous safe-Rust program does not compile: the borrow checker forbids mutating the map (`remove`) while the reference `p` into it is still live, so the defect cannot be written in the first place (error `E0499`):

```
let mut ctxs: HashMap<u64, UeContext> = HashMap::new();
ctxs.insert(1001, UeContext { messages_handled: 0 });
let p = ctxs.get_mut(&1001).unwrap(); // mutable borrow starts
ctxs.remove(&1001);                   // borrow while p live -> rejected
p.messages_handled += 1;              // p used after the remove
```